



บทที่ 5 การสร้างรายงานและแดชบอร์ด

- 5.1 หลักการเลือก Chart ให้เหมาะสมกับข้อมูล
- 5.2 เครื่องมือที่ใช้ในการสร้างรายงานและแดชบอร์ด
- 5.3 การใช้งาน Python Libraries
- 5.4 การใช้งาน Excel
- 5.5 การใช้งาน Power BI
- 5.6 แบบฝึกหัด



5.1 หลักการเลือก Chart ให้เหมาะสมกับข้อมูล

- เลือก Chart ตามวัตถุประสงค์ของการสื่อสาร
- เลือก Chart ให้สอดคล้องกับชนิดของข้อมูล
- หลีกเลี่ยง Chart ที่ทำให้เกิดการตีความผิด
- หลักการใช้สีเพื่อสนับสนุนการอ่านกราฟ

5.1.1 เลือก Chart ตามวัตถุประสงค์ของการสื่อสาร

วัตถุประสงค์	Chart ที่เหมาะสม
เปรียบเทียบค่า	Bar Chart, Column Chart
แสดงแนวโน้มตามเวลา	Line Chart, Area Chart
แสดงสัดส่วน	Pie Chart, Donut Chart
แสดงการกระจายตัว	Box Plot
แสดงการแจกแจง (รูปร่าง)	Histogram
แสดงความสัมพันธ์	Scatter Plot
แสดงโครงสร้าง	Tree Map
แสดงความเข้มข้น (ความถี่)	Heat Map

5.1.2 เลือก Chart ให้สอดคล้องกับชนิดของข้อมูล

ชนิดของข้อมูล	Chart ที่เหมาะสม
ข้อมูลเชิงกลุ่ม (Categorical)	Bar Chart, Column Chart
ข้อมูลเชิงปริมาณ (Numerical)	Histogram, Box Plot
ข้อมูลตามเวลา (Time Series)	Line Chart, Area Chart
ข้อมูลเชิงสัดส่วน (Part-to-whole)	Pie Chart

5.1.3 หลีกเลียง Chart ที่ทำให้เกิดการตีความผิดพลาดหรือยาก

- ไม่ควรใช้ Pie Chart เมื่อค่ามีความใกล้เคียงกันมาก
- ไม่ควรใช้ Pie Chart เมื่อมีจำนวนกลุ่มข้อมูลมากเกินไป (ไม่ควรเกิน 5 กลุ่ม)
- ไม่ควรใช้ 3D Chart เพราะทำให้อ่านยาก
- ไม่ควรใช้ Chat ที่มีหลายแกน (Dual Axis) หากไม่จำเป็น

5.1.4 หลักการใช้สีเพื่อสนับสนุนการอ่านกราฟ

- ใช้สีเดียวกันแทนข้อมูลประเภทเดียวกัน
- ใช้สีเน้นข้อมูลสำคัญ ตัวอย่าง:
 - เป้าหมาย (Target)
 - ค่าผิดปกติ (Outlier)
 - ค่าที่ต้องการให้ตัดสินใจ
- ใช้สีตามบริบทและความหมายสากล
 - เขียว → ดี / ผ่าน / เพิ่มขึ้น
 - แดง → แย่ / ต่ำกว่าเกณฑ์ / ลดลง
 - เหลือง → เตือน / ระวัง
- ใช้สีไม่หลากหลายเกินความจำเป็น
- ใช้สีข้อมูล ตัดกับสีพื้นหลัง

5.2 เครื่องมือที่ใช้ในการสร้างรายงานและแดชบอร์ด

- **Code-based**

Python

- Matplotlib: พื้นฐานที่สุด ควบคุมละเอียด
- Seaborn: สวยงาม ใช้ง่าย เหมาะกับสถิติ
- Plotly: Interactive (zoom, hover)

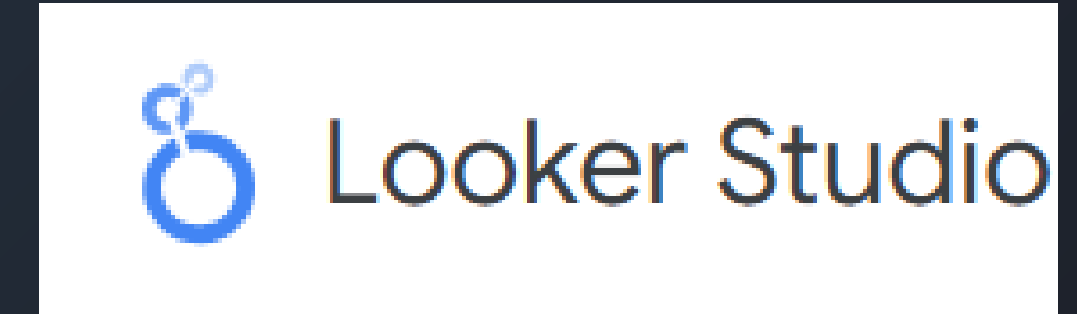
R

- ggplot2: มาตรฐานสายสถิติ
- Shiny: ทำเป็น interactive web

JavaScript

- D3.js: ยืดหยุ่นมาก (ระดับ low-level)
- Chart.js: ใช้ง่าย เหมาะกับเว็บทั่วไป

- **No-code / Low-code**



5.3 การใช้งาน Python Libraries



 plotly

mplfinance

ติดตั้ง Python Libraries

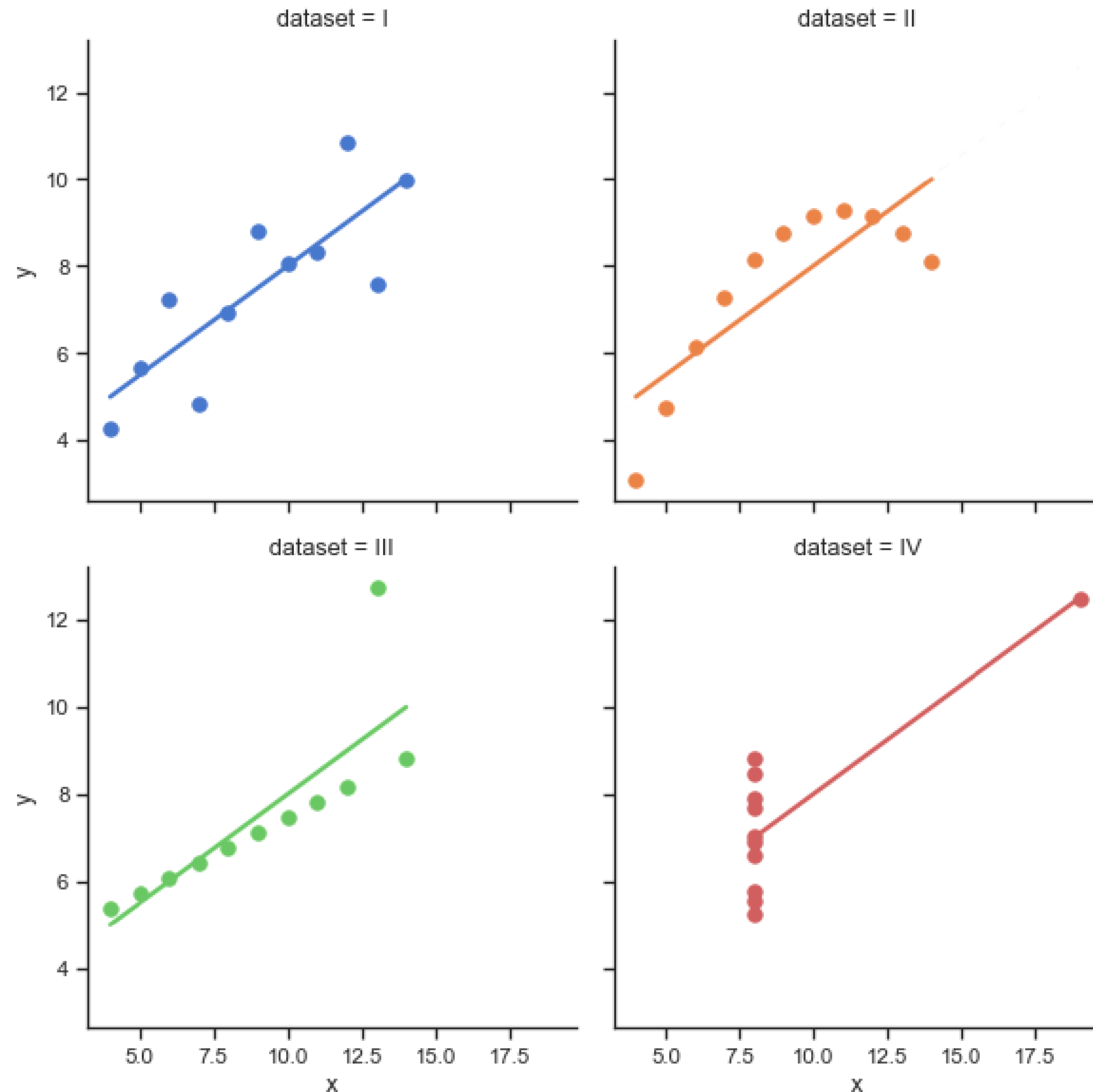
```
1  ### ติดตั้ง Libraries  ###
2
3  #ติดตั้งไลบรารีพื้นฐานสำหรับสร้างกราฟ
4  %pip install matplotlib
5
6  #ติดตั้งไลบรารีสำหรับการสร้างกราฟเชิงสถิติ โดยพัฒนาต่อยอดจาก Matplotlib
7  %pip install seaborn
8
9  #ติดตั้งไลบรารีสำหรับการสร้างกราฟทางการเงิน โดยพัฒนาต่อยอดจาก Matplotlib
10 %pip install mplfinance
11
12 #ติดตั้งไลบรารีสำหรับการสร้างกราฟแบบโต้ตอบ เช่น การ Zoom และแสดงข้อมูลเมื่อคลิกเมาส์
13 %pip install plotly
14
15 #ติดตั้งไลบรารีสำหรับการจัดการไฟล์ Jupyter Notebook และใช้แสดงกราฟจาก Plotly
16 %pip install nbformat
17
18 #ติดตั้งไลบรารีสำหรับดึงข้อมูลราคาหุ้นและข้อมูลทางการเงินจาก Yahoo Finance
19 %pip install yfinance
```

ตั้งค่าฟอนต์

```
1  #Config ฟอนต์
2  #!wget -q https://staff.su.ac.th/Misweb/download/THSarabunNew.zip -O font.zip
3  #!unzip -qj font.zip THSarabunNew.ttf
4  import matplotlib.pyplot as plt
5  from matplotlib import font_manager
6
7  font_manager.fontManager.addfont(r'C:\Windows\Fonts\THSarabunNew.ttf')
8  plt.rcParams['font.family']='TH Sarabun New'
9  plt.rcParams['axes.titlesize'] = 20
10 plt.rcParams['axes.labelsize'] = 20
11 plt.rcParams['xtick.labelsize'] = 20
12 plt.rcParams['ytick.labelsize'] = 20
13 plt.rcParams['legend.fontsize'] = 14
14 plt.rcParams['font.size'] = 20
15 # plt.rcParams['axes.titlecolor']='black'
16 # plt.rcParams['axes.labelcolor']='black'
```

Python

5.3.1 Scatter Plot with Linear Regression Line (Anscombe)



Anscombe คือ ชุดข้อมูลที่ถูกสร้างขึ้นโดยนักสถิติชื่อ Francis Anscombe ในปี 1973 เพื่อแสดงให้เห็นถึงความสำคัญของ Data Visualization แทนที่จะเชื่อเพียง Descriptive Statistics มีทั้งหมด 44 แถว 3 คอลัมน์ ได้แก่

- x: ค่าในแกน x
- y: ค่าในแกน y
- dataset: กลุ่มข้อมูลย่อย ประกอบด้วย
 - **Dataset I:** เป็นข้อมูลที่มีความสัมพันธ์เชิงเส้น (Linear) เหมาะกับ Linear Models
 - **Dataset II:** เป็นข้อมูลที่มีความสัมพันธ์เป็นเส้นโค้ง (Non-linear) การใช้เส้นตรงมาทำนายจึงผิดพลาด
 - **Dataset III:** Linear + Outlier ทำให้ Regression Line เบี่ยงออก
 - **Dataset IV:** ข้อมูลเกือบทั้งหมดเกาะกลุ่มกันที่ค่า x เดียวกัน แต่มี Outlier หนึ่งจุดที่ทำให้ดูเหมือนมีความสัมพันธ์เชิงเส้น

Descriptive Statistics បង្ហាញឱ្យឃើញ Anscombe

	x	y
count	44.000000	44.000000
mean	9.000000	7.500682
std	3.198837	1.958925
min	4.000000	3.100000
25%	7.000000	6.117500
50%	8.000000	7.520000
75%	11.000000	8.747500
max	19.000000	12.740000

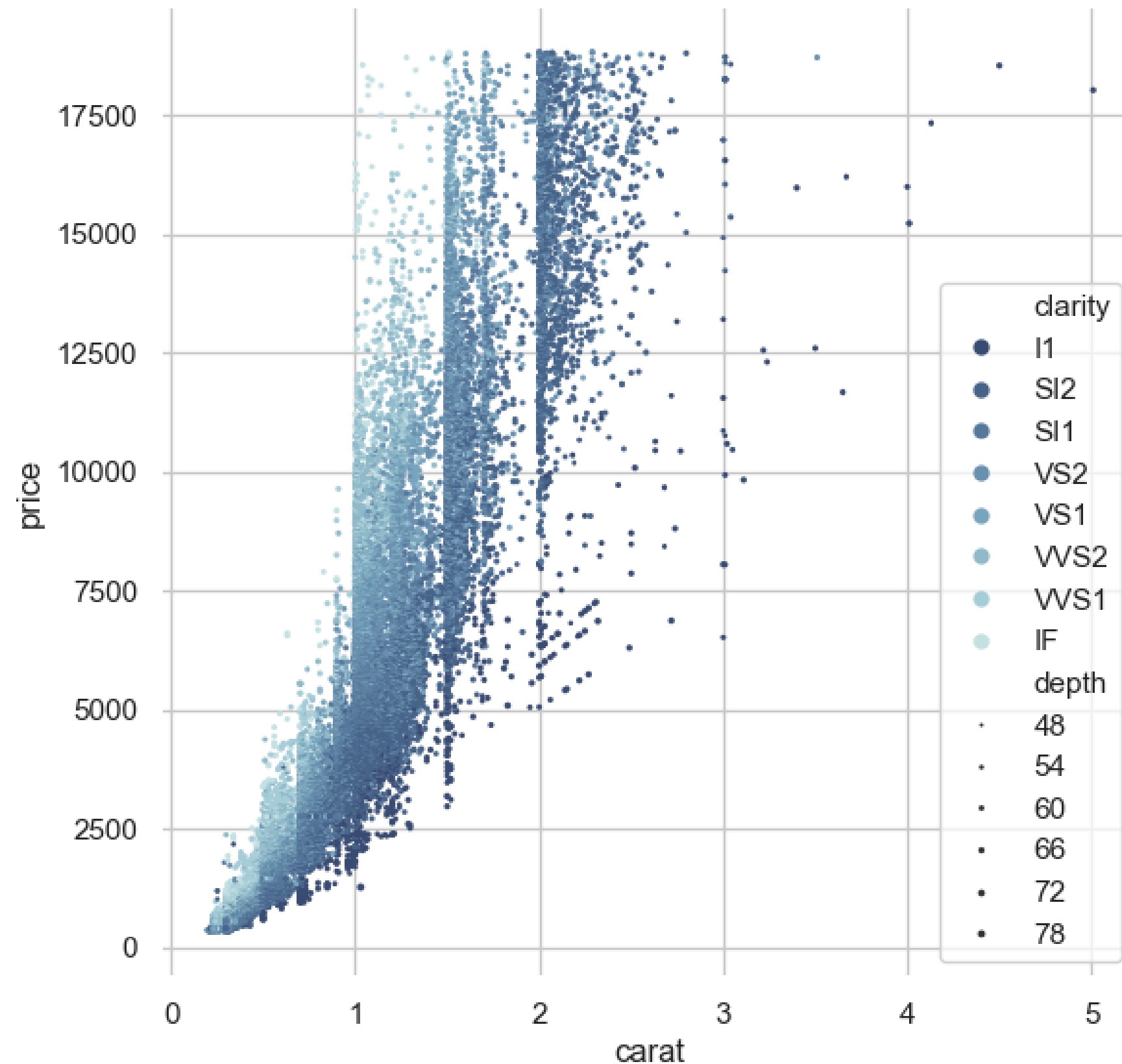
dataset	count	percent
I	11	25.0%
II	11	25.0%
III	11	25.0%
IV	11	25.0%

ไพนอนโค้ด สำหรับ Scatter Plot with Linear Regression Line (Anscombe)

```
1 import seaborn as sns
2 sns.set_theme(style="ticks")
3
4 # Load the example dataset for Anscombe's quartet
5 df = sns.load_dataset("anscombe")
6
7 # Show the results of a linear regression within each dataset
8 sns.lmplot(
9     data=df, x="x", y="y", col="dataset", hue="dataset",
10    col_wrap=2, palette="muted", ci=None,
11    height=4, scatter_kws={"s": 50, "alpha": 1}
12 )
```

- `sns.set_theme(style="ticks")`: สร้างขีด (ticks) บนแกน X และ Y เพื่อให้ดูค่า Scale ของข้อมูลได้ง่าย
`style = {darkgrid, whitegrid, dark, white, ticks}`
- `df = sns.load_dataset("anscombe")`: โหลดชุดข้อมูล “anscombe” ซึ่งเป็นชุดข้อมูลตัวอย่างของ Seaborn มี 3 คอลัมน์ ได้แก่ x, y และคอลัมน์ dataset (ที่ระบุว่าเป็นกลุ่ม I, II, III หรือ IV) ทั้งหมด 44 แถว
- `sns.lmplot`: เป็นฟังก์ชันที่ใช้สร้าง Scatter Plot พร้อมวาด Regression Line โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `data=df`: กำหนดข้อมูลที่จะนำมาสร้าง Scatter Plot
 - `x="x"`: กำหนดคอลัมน์ที่นำมาสร้างกราฟในแนวแกน x
 - `y="y"`: กำหนดคอลัมน์ที่นำมาสร้างกราฟในแนวแกน y
 - `col="dataset"`: สั่งให้แยกกราฟตามชื่อกลุ่ม (I, II, III, IV) ออกเป็นคอลัมน์ละคอลัมน์
 - `hue="dataset"`: สั่งให้ใช้ "สี" ที่ต่างกันในแต่ละชุดข้อมูล เพื่อให้แยกแยะด้วยสายตาได้ง่าย
 - `col_wrap=2`: กำหนดให้แสดงผลคอลัมน์ละ 2 รูป (รวม 2 แถว จะได้ครบ 4 ชุดข้อมูลพอดี)
 - `palette="muted"`: กำหนดโทนสี {deep, muted, bright, pastel, dark, colorblind}
 - `ci=None`: ปิดการแสดงความเชื่อมั่น (Confidence Interval) หรือแถบเงา ๆ รอบเส้นตรง เพื่อให้เห็น Regression Line ชัดเจน
 - `scatter_kws={"s": 50, "alpha": 1}`: ปรับขนาดของจุดข้อมูล (size=50) และความโปร่งแสง (alpha=1 คือทึบแสง)

5.3.2 Scatter Plot with Multiple Semantics (Diamonds)



Diamonds คือ ชุดข้อมูลที่ประกอบด้วยข้อมูลราคา และคุณลักษณะต่าง ๆ ของเพชร มีทั้งหมด 53,940 แถว 10 คอลัมน์ แบ่งออกเป็น 3 กลุ่ม ดังนี้

- ปัจจัยหลัก (หัวใจสำคัญของราคาเพชร)
 - carat (กะรัต) 1 กะรัต = 0.2 กรัม
 - cut (คุณภาพการเจียรระโน)
 - color (สี)
 - clarity (ความสะอาด): ตำแหน่งภายในเพชร เรียงจากแย่ไปดี: I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF
- มิติและสัดส่วน
 - x: ความยาว (มม.)
 - y: ความกว้าง (มม.)
 - z: ความลึก (มม.)
 - depth: เปอร์เซ็นต์ความลึก
 - table: ความกว้างของหน้าเพชร
- ตัวแปรเป้าหมาย (Target Variable)
 - price: ราคา (\$)

Descriptive Statistics ဗဝ်းပုဂံမုာ် Diamonds

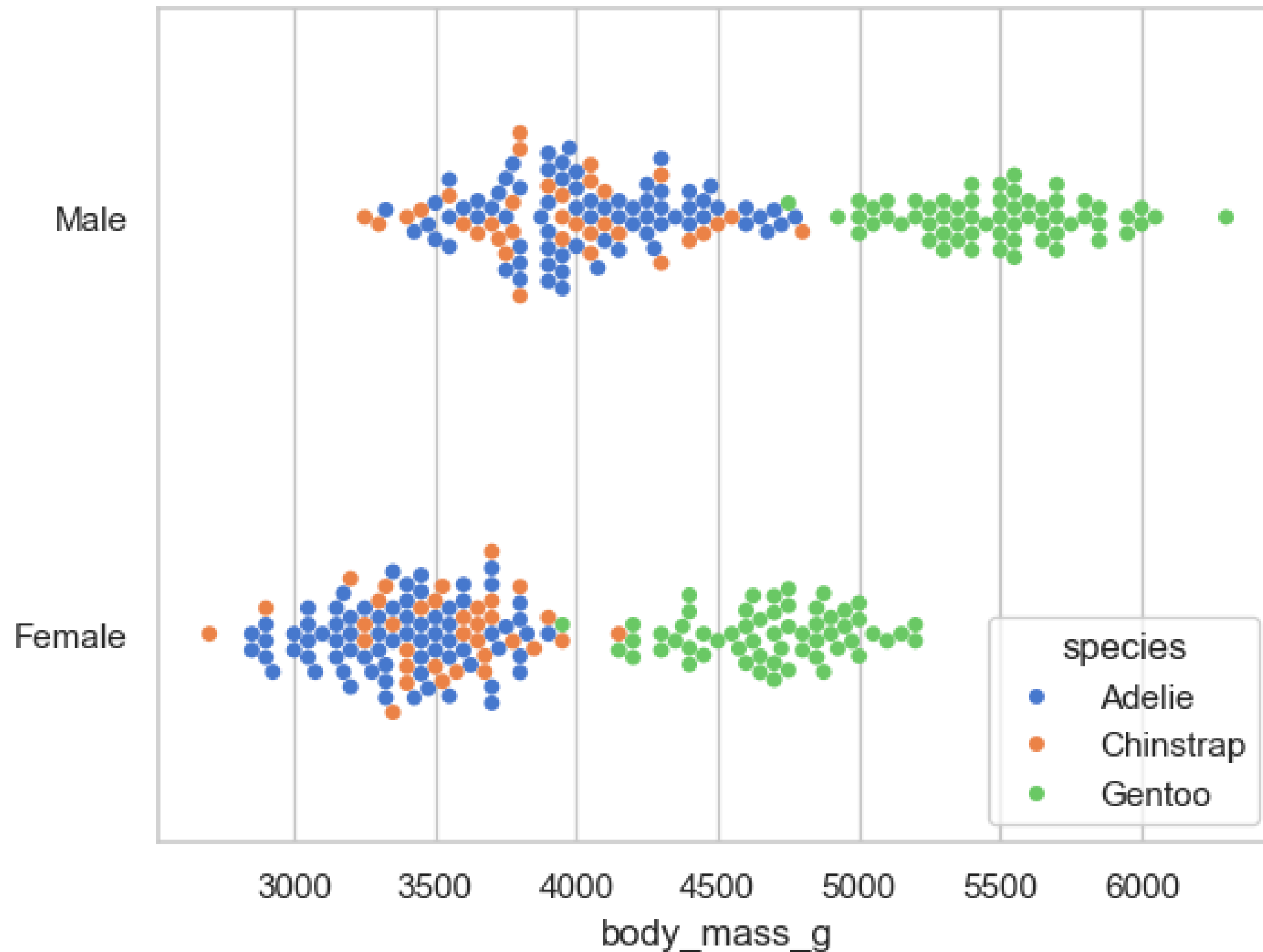
	carat	depth	table	price	x	y	z	cut	count	percent
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000	Ideal	21551	39.95%
mean	0.797940	61.749405	57.457184	3932.799722	5.731157	5.734526	3.538734	Premium	13791	25.57%
std	0.474011	1.432621	2.234491	3989.439738	1.121761	1.142135	0.705699	Very Good	12082	22.4%
min	0.200000	43.000000	43.000000	326.000000	0.000000	0.000000	0.000000	Good	4906	9.1%
25%	0.400000	61.000000	56.000000	950.000000	4.710000	4.720000	2.910000	Fair	1610	2.98%
50%	0.700000	61.800000	57.000000	2401.000000	5.700000	5.710000	3.530000			
75%	1.040000	62.500000	59.000000	5324.250000	6.540000	6.540000	4.040000			
max	5.010000	79.000000	95.000000	18823.000000	10.740000	58.900000	31.800000			
color	count	percent	clarity	count	percent					
D	6775	12.56%	IF	1790	3.32%					
E	9797	18.16%	VVS1	3655	6.78%					
F	9542	17.69%	VVS2	5066	9.39%					
G	11292	20.93%	VS1	8171	15.15%					
H	8304	15.39%	VS2	12258	22.73%					
I	5422	10.05%	SI1	13065	24.22%					
J	2808	5.21%	SI2	9194	17.04%					
			I1	741	1.37%					

ไพทอนโค้ด สำหรับ Scatter Plot with Multiple Semantics (Diamonds)

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 sns.set_theme(style="whitegrid")
4
5 # Load the example diamonds dataset
6 diamonds = sns.load_dataset("diamonds")
7
8 # Draw a scatter plot while assigning point colors and sizes to different
9 # variables in the dataset
10 f, ax = plt.subplots(figsize=(6.5, 6.5))
11 sns.despine(f, left=True, bottom=True)
12 clarity_ranking = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]
13 sns.scatterplot(x="carat", y="price",
14                 hue="clarity", size="depth",
15                 palette="ch:r=-.2,d=.3_r",
16                 hue_order=clarity_ranking,
17                 sizes=(1, 8), linewidth=0,
18                 data=diamonds, ax=ax)
```

- `f, ax = plt.subplots(figsize=(6.5, 6.5))`: กำหนดขนาดกว้าง และความยาวของภาพ เท่ากันที่ 6.5 นิ้ว
- `sns.despine(f, left=True, bottom=True)`: เอาเส้นขอบ (Spines) ด้านซ้ายและด้านล่างออก ทำให้กราฟดู "ลอย" และ กันสมัยขึ้น (Minimalist)
- `clarity_ranking`: กำหนดลำดับของความสะอาดของเพชร (Clarity) จากน้อยไปมาก (I1 คือ ตำหนิเยอะสุด ไปจนถึง IF คือ ไร้ตำหนิ) การทำแบบนี้เพื่อให้ใน Legend (คำอธิบายสัญลักษณ์) และ การไล่สี เรียงลำดับตามคุณภาพจริง ไม่ใช่เรียงตามตัวอักษร
- `sns.scatterplot`: เป็นฟังก์ชันที่ใช้สร้าง Scatter Plot โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `hue_order=clarity_ranking`: กำหนดความเข้มของสี โดยอ้างอิงลำดับจาก `clarity_ranking`
 - `size="depth"`: ใช้ความลึกของเพชร (Depth %) แทน ขนาดของจุด
 - `palette="ch:r=-.2,d=.3_r"`:
 - `ch` (Cubehelix): ใช้ระบบสี Cubehelix (ระบบเฉดสีรอบวงล้อสี)
 - `r=-.2` : กำหนดทิศทางการ "หมุน" (Rotation) ค่า - จะหมุนย้อนเข็มนาฬิกา (มักจะได้โทน เขียว/ฟ้า/ม่วง) ค่า + จะหมุนไปทางแดง/เหลือง
 - `d=.3`: กำหนดความเข้มขึ้นของสีหรือความสด (Darkness/Hue Saturation) ค่า .3 จะให้สีที่ค่อนข้างหม่น และดูเป็นธรรมชาติ ไม่ดูฉูดฉาดเกินไป เหมาะกับข้อมูลจำนวนมากอย่าง Diamonds dataset เพื่อไม่ให้ลายตา
 - `_r`: หมายถึงการ "กลับด้าน" ของลำดับสี (Reverse) ปกติ Cubehelix จะเริ่มจากมืดไปสว่าง แต่เมื่อใส่ `_r` จะเปลี่ยนจาก สว่างไปมืด
 - `sizes=(1, 8)`: กำหนดขอบเขตขนาดของจุดให้มีขนาดตั้งแต่ 1 ถึง 8 หน่วย เพื่อไม่ให้จุดใหญ่จนบังกันมิด
 - `linewidth=0`: เอาเส้นขอบรอบจุดออก เพื่อให้กลุ่มข้อมูลที่หนาแน่นดูเนียนตา ไม่ขัดตากับเส้นขอบจุด

5.3.3 Scatter Plot with Categorical Variables (Penguins)



Penguins คือ ชุดข้อมูลนกเพนกวิน มีทั้งหมด 344 แถว 7 คอลัมน์ ได้แก่

- Categorical Data
 - **species**: สายพันธุ์นกเพนกวิน (Adelie, Chinstrap, Gentoo)
 - island: เกาะที่พบนกเพนกวิน
 - **sex**: เพศ (Male, Female)
- Numerical Data
 - bill_length_mm: ความยาวปาก (มม.)
 - bill_depth_mm: ความหนาปาก (มม.)
 - flipper_length_mm: ความยาวครีบ (มม.)
 - **body_mass_g**: น้ำหนักตัว (กรัม)



Descriptive Statistics ของชุดข้อมูล Penguins

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	190.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000

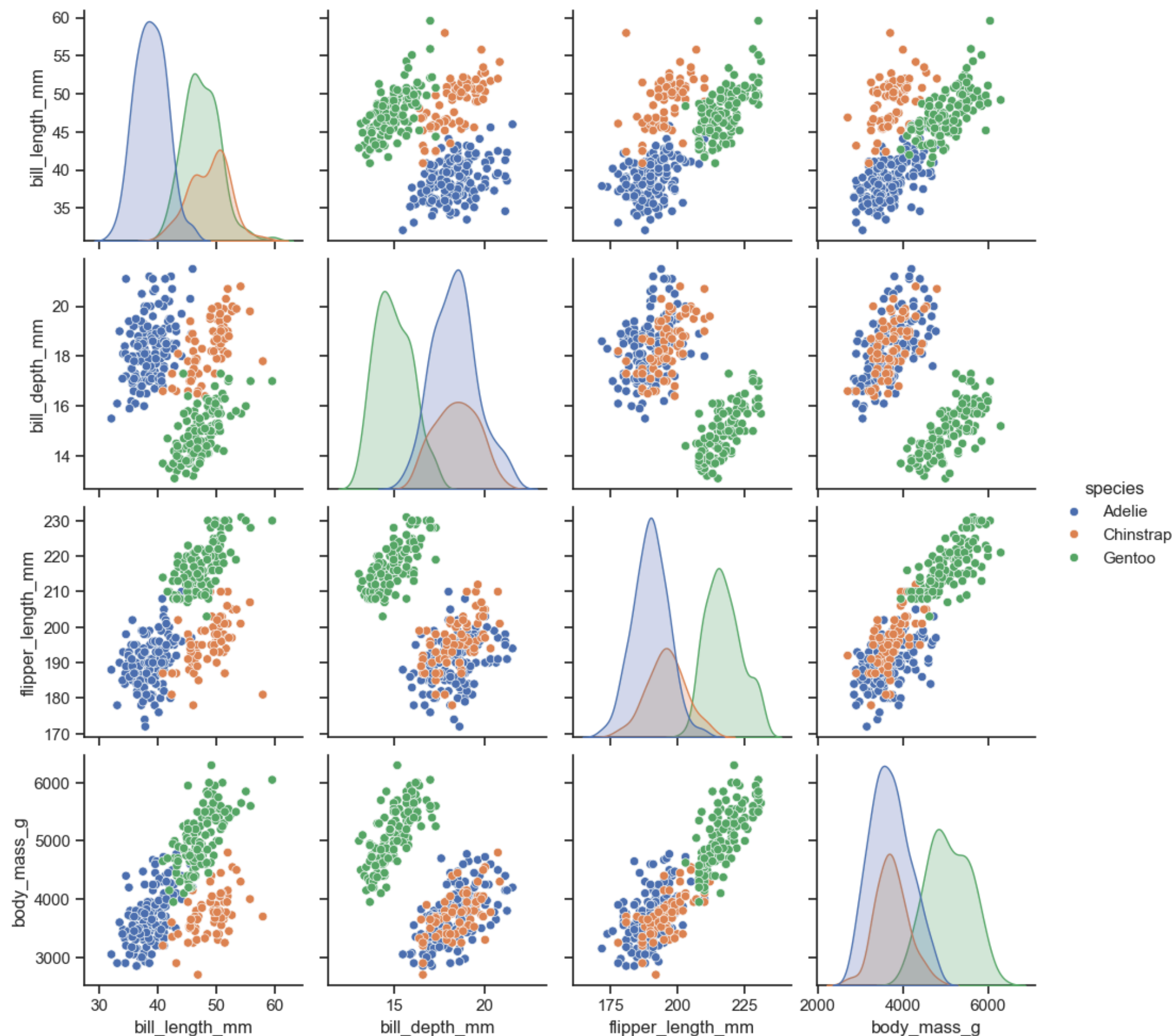
species	count	percent
Adelie	152	44.19%
Chinstrap	68	19.77%
Gentoo	124	36.05%
island	count	percent
Biscoe	168	48.84%
Dream	124	36.05%
Torgersen	52	15.12%
sex	count	percent
Female	165	49.55%
Male	168	50.45%

ไพล์โค้ด สำหรับ Scatter Plot with Categorical Variables (Penguins)

```
1 import seaborn as sns
2 sns.set_theme(style="whitegrid", palette="muted")
3
4 # Load the penguins dataset
5 df = sns.load_dataset("penguins")
6
7 # Draw a categorical scatterplot to show each observation
8 ax = sns.swarmplot(data=df, x="body_mass_g", y="sex", hue="species")
9 ax.set(ylabel="")
```

- `sns.swarmplot`: เป็นการสร้าง Swarm Plot ซึ่งเป็นรูปแบบหนึ่งของ Scatter Plot ที่ใช้แสดงการกระจายตัวของข้อมูลเชิงปริมาณ (Quantitative/Numerical) แยกตามหมวดหมู่ (Categorical)

5.3.4 Scatter Plot Matrix (Penguins)



Penguins คือ ชุดข้อมูลนกเพนกวิน มีทั้งหมด 344 แถว 7 คอลัมน์ ได้แก่

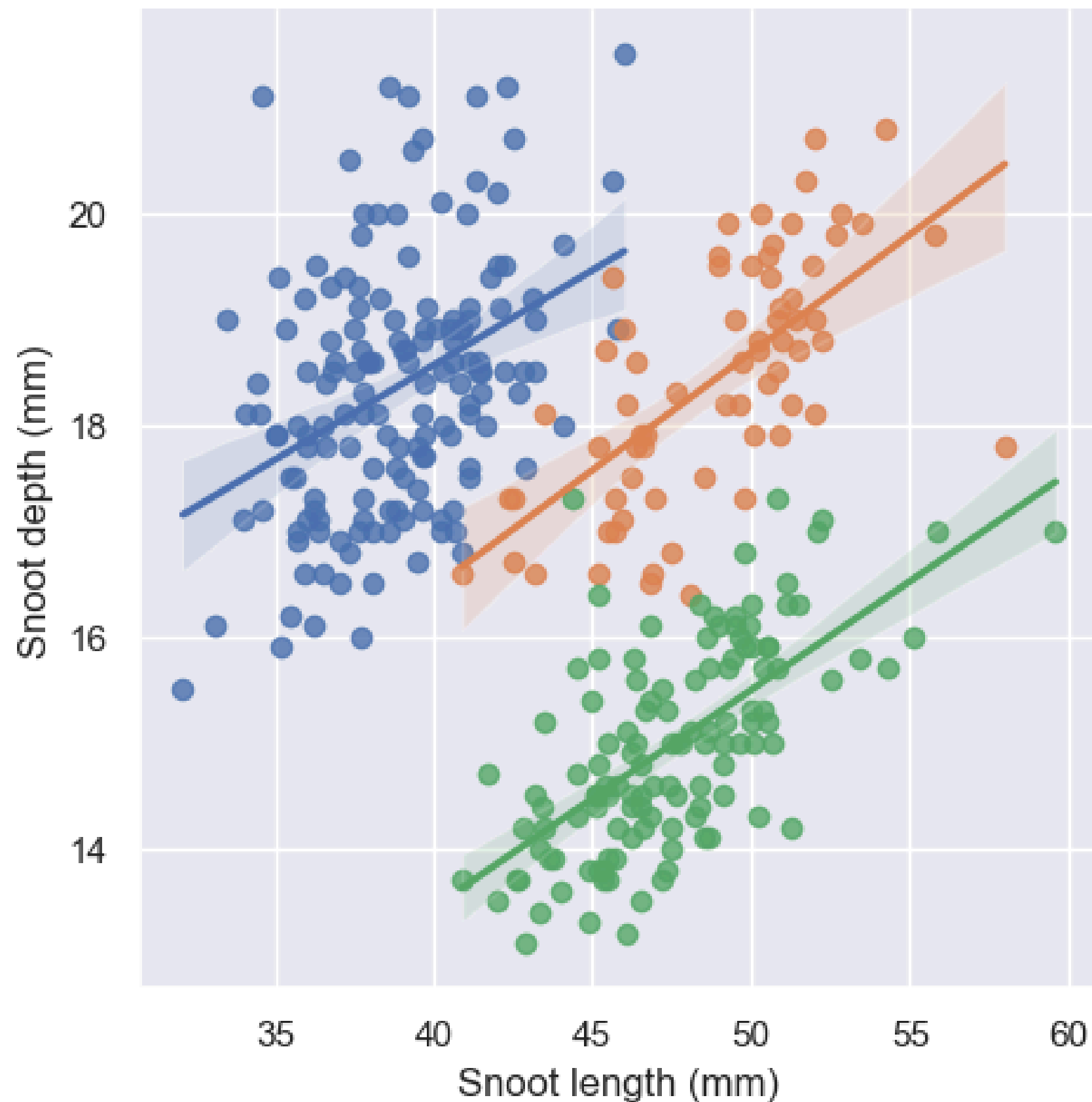
- Categorical Data
 - species: สายพันธุ์นกเพนกวิน (Adelie, Chinstrap, Gentoo)
 - island: เกาะที่พบนกเพนกวิน
 - sex: เพศ (Male, Female)
- Numerical Data
 - **bill_length_mm**: ความยาวปาก (มม.)
 - **bill_depth_mm**: ความหนาปาก (มม.)
 - **flipper_length_mm**: ความยาวครีบ (มม.)
 - **body_mass_g**: น้ำหนักตัว (กรัม)

ไพล์โค้ด สำหรับ Scatter Plot Matrix (Penguins)

```
1 import seaborn as sns
2 sns.set_theme(style="ticks")
3
4 df = sns.load_dataset("penguins")
5 sns.pairplot(df, hue="species")
```

- `sns.pairplot`: เป็นการสร้าง Pair Plot ซึ่งเป็นรูปแบบหนึ่งของ Scatter Plot ที่ใช้แสดงความสัมพันธ์ระหว่างตัวแปรทุกคู่พร้อมกัน

5.3.5 Scatter Plot with Multiple Linear Regression (Penguins)



Penguins คือ ชุดข้อมูลนกเพนกวิน มีทั้งหมด 344 แถว 7 คอลัมน์ ได้แก่

- Categorical Data
 - species: สายพันธุ์นกเพนกวิน (Adelie, Chinstrap, Gentoo)
 - island: เกาะที่พบนกเพนกวิน
 - sex: เพศ (Male, Female)
- Numerical Data
 - bill_length_mm: ความยาวปาก (มม.)
 - bill_depth_mm: ความหนาปาก (มม.)
 - flipper_length_mm: ความยาวครีบ (มม.)
 - body_mass_g: น้ำหนักตัว (กรัม)

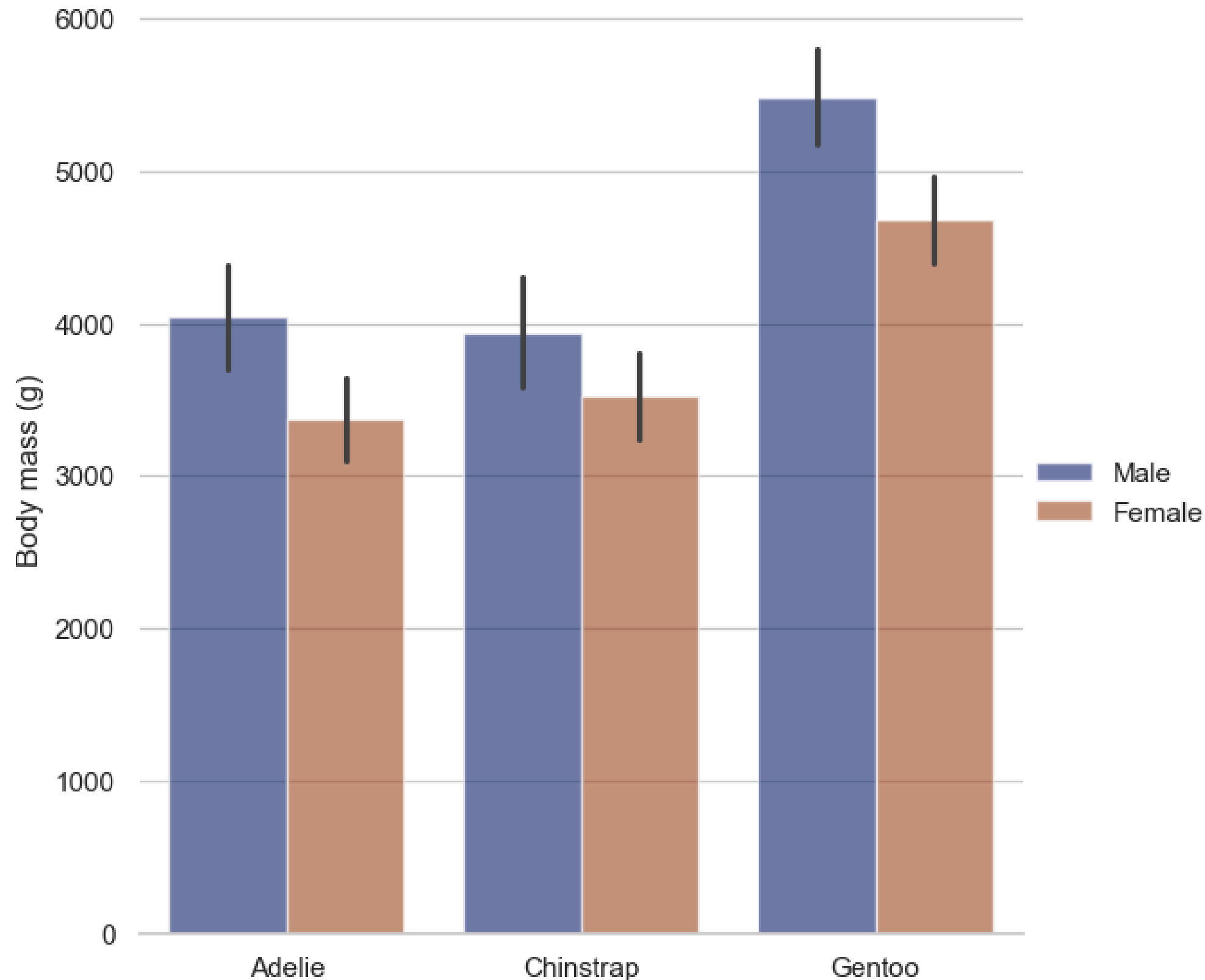


ไพล์โค้ด สำหรับ Scatter Plot with Multiple Linear Regression (Penguins)

```
1 import seaborn as sns
2 sns.set_theme()
3
4 # Load the penguins dataset
5 penguins = sns.load_dataset("penguins")
6
7 # Plot sepal width as a function of sepal_length across days
8 g = sns.lmplot(
9     data=penguins,
10    x="bill_length_mm", y="bill_depth_mm", hue="species",
11    height=5
12 )
13
14 # Use more informative axis labels than are provided by default
15 g.set_axis_labels("Snoot length (mm)", "Snoot depth (mm)")
```

- `sns.lmplot`: เป็นการสร้าง Linear Model Plot ซึ่งเป็นรูปแบบหนึ่งของ Scatter Plot ที่ใช้ในการวาด Linear Regression Line และ Confidence Interval

5.3.6 Grouped Bar Plots (Penguins)



Penguins คือ ชุดข้อมูลนกเพนกวิน มีทั้งหมด 344 แถว 7 คอลัมน์ ได้แก่

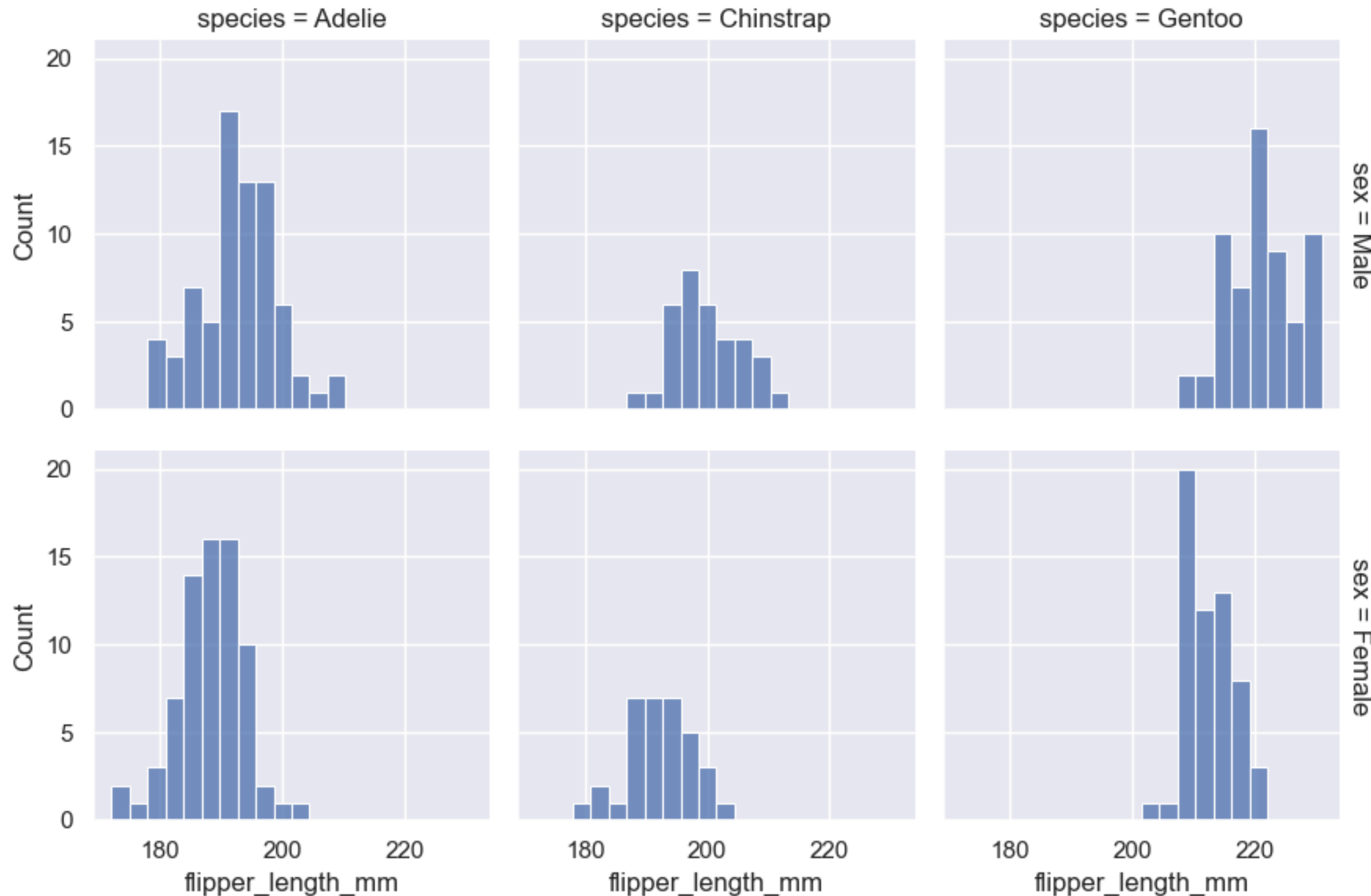
- Categorical Data
 - species: สายพันธุ์นกเพนกวิน (Adelie, Chinstrap, Gentoo)
 - island: เกาะที่พบนกเพนกวิน
 - sex: เพศ (Male, Female)
- Numerical Data
 - bill_length_mm: ความยาวปาก (มม.)
 - bill_depth_mm: ความหนาปาก (มม.)
 - flipper_length_mm: ความยาวครีบ (มม.)
 - body_mass_g: น้ำหนักตัว (กรัม)

ไพล์โค้ด สำหรับ Grouped Barplots (Penguins)

```
1 import seaborn as sns
2 sns.set_theme(style="whitegrid")
3
4 penguins = sns.load_dataset("penguins")
5
6 # Draw a nested barplot by species and sex
7 g = sns.catplot(
8     data=penguins, kind="bar",
9     x="species", y="body_mass_g", hue="sex",
10    errorbar="sd", palette="dark", alpha=.6, height=6
11 )
12 g.despine(left=True)
13 g.set_axis_labels("", "Body mass (g)")
14 g.legend.set_title("")
```

- `sns.catplot`: เป็นฟังก์ชันที่ใช้สร้าง Categorical (Bar) Plot โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `kind="bar"`: กำหนดประเภทกราฟเป็น กราฟแท่ง ซึ่งจะคำนวณ "ค่าเฉลี่ย" (Mean) ให้โดยอัตโนมัติ
 - `errorbar="sd"`: แสดงเส้นขีดบนแท่งกราฟ โดยใช้ค่า SD เพื่อให้เห็นการกระจายตัวของน้ำหนักในแต่ละกลุ่ม
 - `g.despine(left=True)`: เอาเส้นขอบแนวตั้งด้านซ้ายออก
 - `g.set_axis_labels("", "Body mass (g)")`: แสดงเฉพาะ Label บนแกน y
 - `g.legend.set_title("")`: ลบ Title ของ Legend ออก

5.3.7 Histograms by Subsets of Data (Penguins)



Penguins คือ ชุดข้อมูลนกเพนกวิน มีทั้งหมด 344 แถว 7 คอลัมน์ ได้แก่

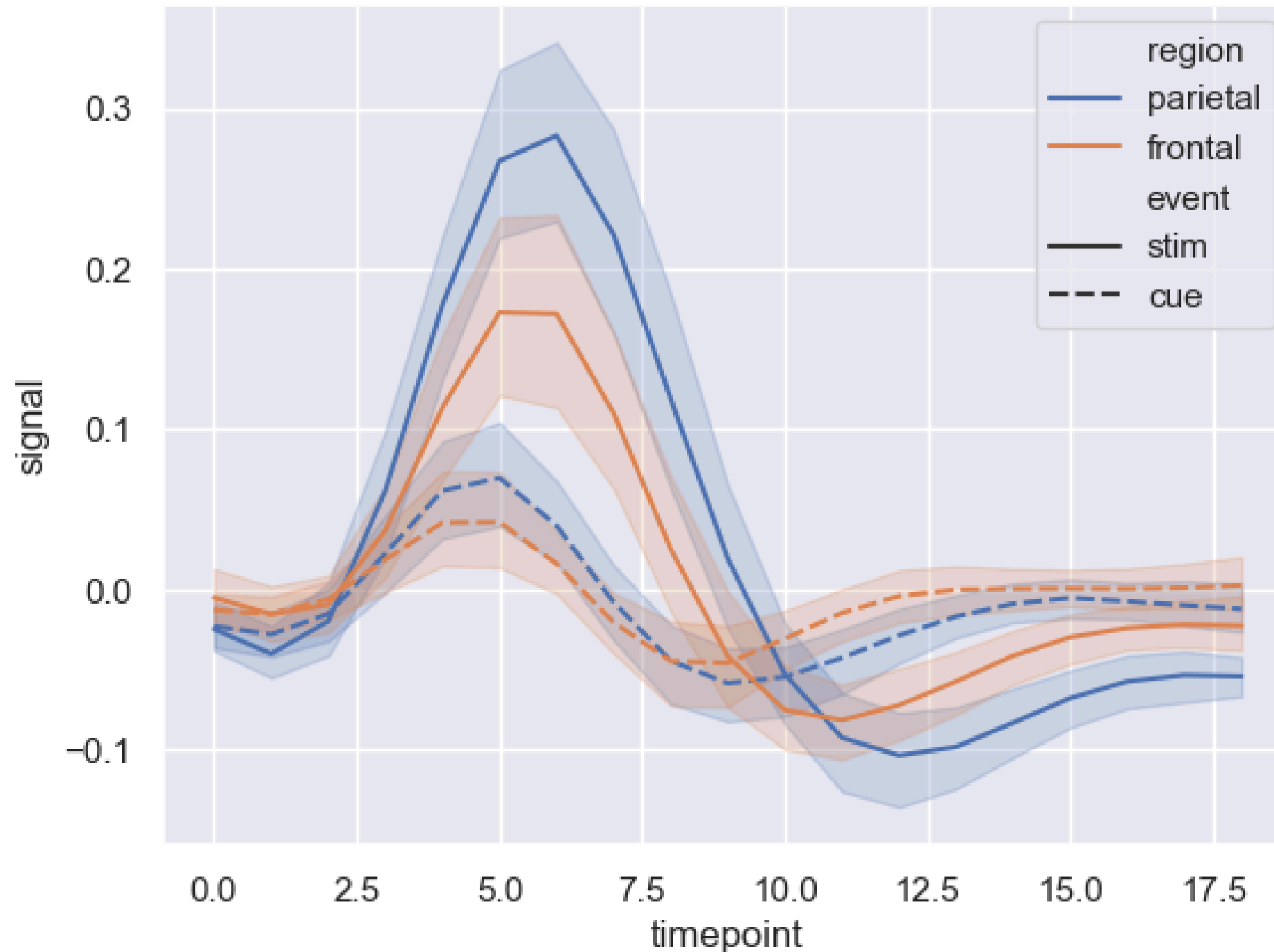
- Categorical Data
 - species: สายพันธุ์นกเพนกวิน (Adelie, Chinstrap, Gentoo)
 - island: เกาะที่พบนกเพนกวิน
 - sex: เพศ (Male, Female)
- Numerical Data
 - bill_length_mm: ความยาวปาก (มม.)
 - bill_depth_mm: ความหนาปาก (มม.)
 - flipper_length_mm: ความยาวครีบ (มม.)
 - body_mass_g: น้ำหนักตัว (กรัม)

ไพล์โค้ด สำหรับ Histograms by Subsets of Data (Penguins)

```
1 import seaborn as sns
2
3 sns.set_theme(style="darkgrid")
4 df = sns.load_dataset("penguins")
5 sns.displot(
6     df, x="flipper_length_mm", col="species", row="sex",
7     binwidth=3, height=3, facet_kws=dict(margin_titles=True),
8 )
```

- `sns.displot`: เป็นฟังก์ชันที่ใช้สร้าง Distribution Plot เพื่อแสดง Histogram ในแต่ละกลุ่มข้อมูล โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `col="species"`: แยกกราฟออกเป็น คอลัมน์ ตามสายพันธุ์ (3 คอลัมน์)
 - `row="sex"`: แยกกราฟออกเป็น แถว ตามเพศ (2 แถว)
 - `binwidth=3`: กำหนดความกว้างของแท่ง Histogram ให้ครอบคลุมช่วงละ 3 มม. ช่วยให้เห็นความละเอียดของการกระจายตัวของข้อมูลได้ดีขึ้น
 - `height=3`: กำหนดความสูงของแต่ละกราฟ (Subplot) เป็น 3 นิ้ว
 - `facet_kws=dict(margin_titles=True)`: ทำให้ชื่อของแถว (เช่น sex: Male) ไปแสดงที่ขอบด้านขวาของกราฟ แทนที่จะแสดงทับอยู่บนกราฟแต่ละรูป ทำให้ดูสะอาดตาขึ้น

5.3.8 Line Plot with Error Bands (fMRI)



fMRI ย่อมาจาก **Functional Magnetic Resonance Imaging** คือ ชุดข้อมูลที่ได้จากการสแกนสมองเพื่อวัดการทำงานของสมอง โดยตรวจจับการเปลี่ยนแปลงของการไหลเวียนเลือดและระดับออกซิเจนในสมอง มีทั้งหมด 1,064 แถว 5 คอลัมน์ ได้แก่

- timepoint: เวลาที่บันทึกข้อมูล (0 ถึง 18 วินาที)
- signal: ค่าสัญญาณความเข้มข้นของออกซิเจนในเลือด
- subject: รหัสของผู้ถูกทดสอบ (มี 14 คน: s0 ถึง s13)
- event: ประเภทการกระตุ้น ได้แก่ stim (กระตุ้นต่อสิ่งเร้าจริง) และ cue (สัญญาณเตือน)
- region: บริเวณของสมองที่ตรวจวัด ได้แก่ frontal (หน้าผาก) และ parietal (ข้างขม่อม)



Descriptive Statistics បង្ហាញឱ្យឃើញ fMRI

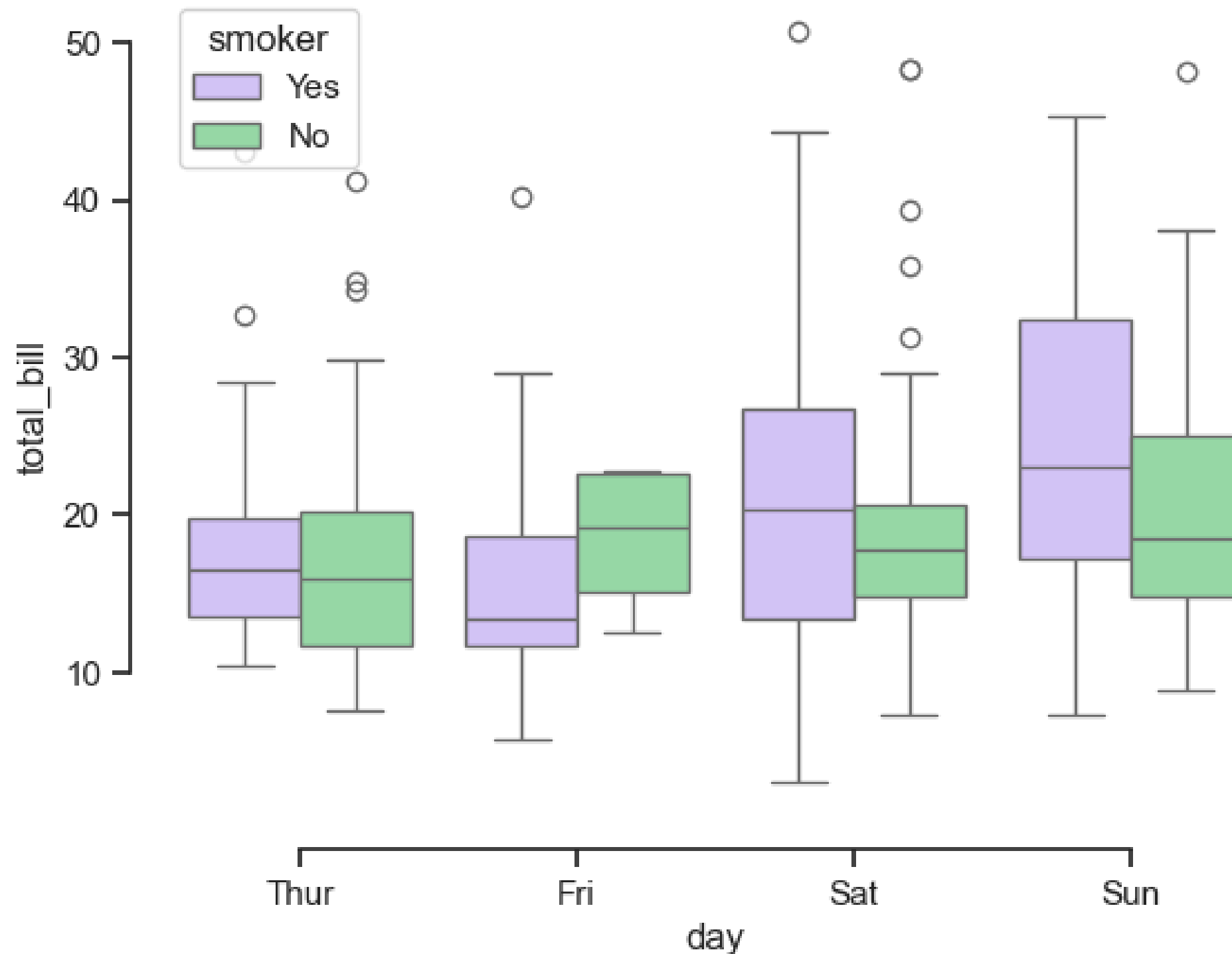
timepoint signal			subject count percent			event count percent		
count	1064.000000	1064.000000	s0	76	7.14%	cue	532	50.0%
mean	9.000000	0.003540	s1	76	7.14%	stim	532	50.0%
std	5.479801	0.093930	s10	76	7.14%			
min	0.000000	-0.255486	s11	76	7.14%	region	count	percent
25%	4.000000	-0.046070	s12	76	7.14%	frontal	532	50.0%
50%	9.000000	-0.013653	s13	76	7.14%	parietal	532	50.0%
75%	14.000000	0.024293	s2	76	7.14%			
max	18.000000	0.564985	s3	76	7.14%			
			s4	76	7.14%			
			s5	76	7.14%			
			s6	76	7.14%			
			s7	76	7.14%			
			s8	76	7.14%			
			s9	76	7.14%			

ไพล็อตสำหรับ Line Plot with Error Bands (fMRI)

```
1 import seaborn as sns
2 sns.set_theme(style="darkgrid")
3
4 # Load an example dataset with long-form data
5 fmri = sns.load_dataset("fmri")
6
7 # Plot the responses for different events and regions
8 sns.lineplot(x="timepoint", y="signal",
9             hue="region", style="event",
10            data=fmri)
```

- `sns.lineplot`: เป็นฟังก์ชันที่ใช้สร้าง Line Plot โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `hue="region"`: แยก "สี" ตามบริเวณสมอง
 - `style="event"`: แยก "ลักษณะเส้น" (เส้นทึบ vs เส้นประ) ตามประเภทการกระตุ้น

5.3.9 Grouped Box Plots (Tips)



Tips คือ ชุดข้อมูลการให้ทิปย์ในร้านอาหารแห่งหนึ่ง มีทั้งหมด 244 แถว 7 คอลัมน์ ได้แก่

- total_bill: ค่าอาหารทั้งหมด (ดอลลาร์)
- tip: จำนวนเงินทิปย์ที่ให้ (ดอลลาร์)
- sex: เพศของผู้จ่ายเงิน (Male, Female)
- smoker: ในโต๊ะนั้นมีคนสูบบุหรี่หรือไม่ (Yes, No)
- day: วันที่ไปรับประทานอาหาร (Thur, Fri, Sat, Sun)
- time: ช่วงเวลา (Lunch, Dinner)
- size: จำนวนคนในโต๊ะนั้น

Descriptive Statistics សង្ខេបຂໍ້ມູນ Tips

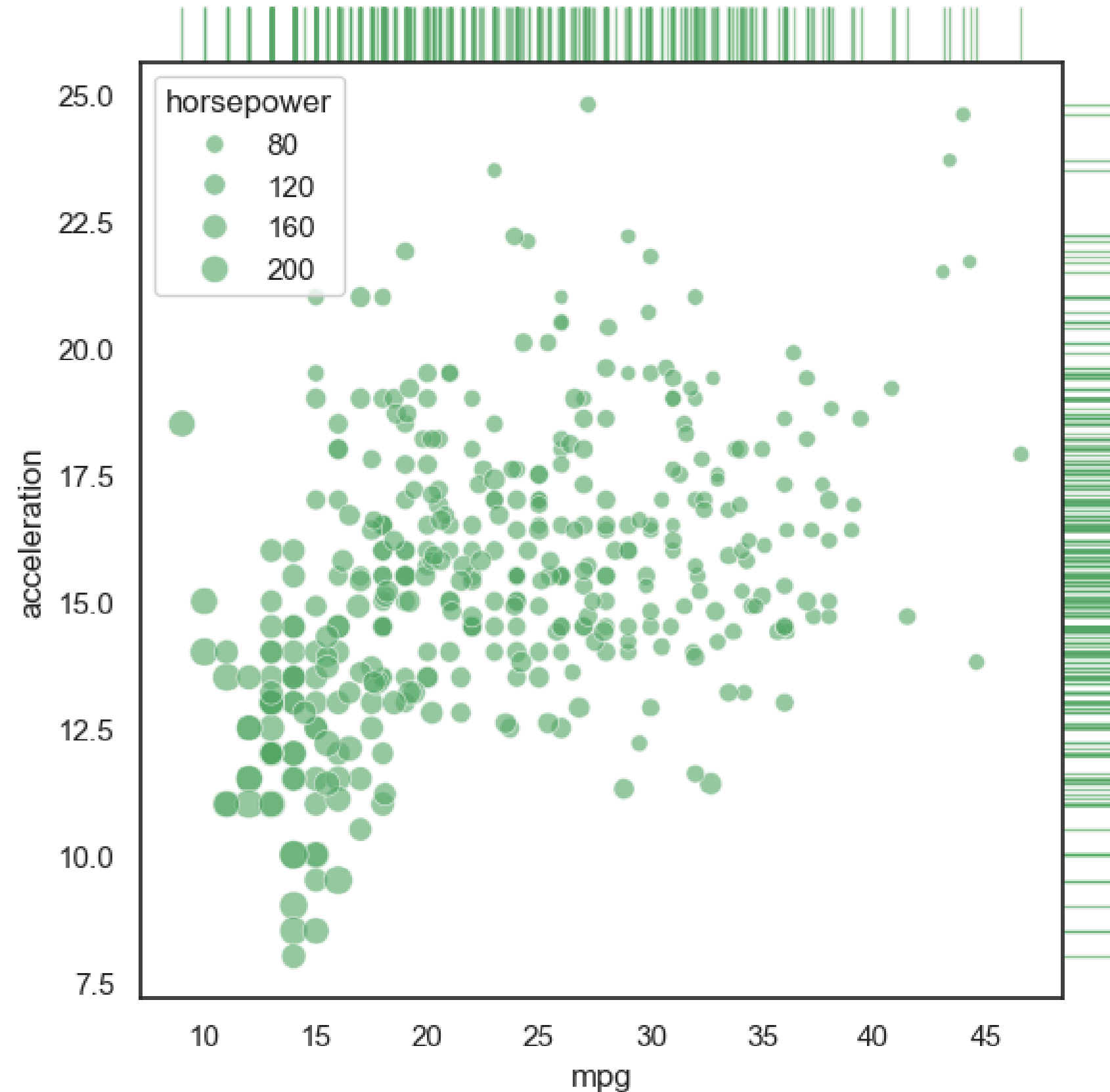
	total_bill	tip	size	sex	count	percent	day	count	percent
count	244.000000	244.000000	244.000000	Male	157	64.34%	Thur	62	25.41%
mean	19.785943	2.998279	2.569672	Female	87	35.66%	Fri	19	7.79%
std	8.902412	1.383638	0.951100				Sat	87	35.66%
min	3.070000	1.000000	1.000000	smoker	count	percent	Sun	76	31.15%
25%	13.347500	2.000000	2.000000	Yes	93	38.11%	time	count	percent
50%	17.795000	2.900000	2.000000	No	151	61.89%	Lunch	68	27.87%
75%	24.127500	3.562500	3.000000				Dinner	176	72.13%
max	50.810000	10.000000	6.000000						

ไพลทอนโค้ด สำหรับ Grouped Box Plots (Tips)

```
1 import seaborn as sns
2 sns.set_theme(style="ticks", palette="pastel")
3
4 # Load the example tips dataset
5 tips = sns.load_dataset("tips")
6
7 # Draw a nested boxplot to show bills by day and time
8 sns.boxplot(x="day", y="total_bill",
9             hue="smoker", palette=["m", "g"],
10            data=tips)
11 sns.despine(offset=10, trim=True)
```

- `sns.boxplot`: เป็นฟังก์ชันที่ใช้สร้าง Boxplot เพื่อแสดงค่าสถิติทั้ง 6 ค่า (Min, Q1, Median, Q3, IQR, Max) และแสดง Outliers โดยมีพารามิเตอร์หลัก ๆ ดังนี้
 - `palette=["m", "g"]`: กำหนดสีด้วยรหัสสีที่เป็นตัวอักษร (m = magenta, g = green)
- `sns.despine(offset=10, trim=True)`:
 - `offset=10`: ดันเส้นแกน X และ Y ให้ห่างจากตัวกราฟเล็กน้อย ทำให้ดูไม่แออัด
 - `trim=True`: ตัดเส้นแกนให้ยาวเท่ากับช่วงข้อมูลที่มีอยู่จริง ทำให้กราฟดู Minimal

5.3.10 Scatter Plot with Marginal Ticks (mpg)



mpg คือ ชุดข้อมูลประสิทธิภาพการใช้น้ำมันของรถยนต์
มีทั้งหมด 398 แถว 9 คอลัมน์ ได้แก่

- mpg: ระยะทางที่วิ่งได้ ไมล์ ต่อน้ำมัน 1 แกลลอน
- cylinders: จำนวนลูกสูบ
- displacement: ปริมาตรกระบอกสูบ (ขนาดเครื่องยนต์)
- horsepower: กำลังแรงม้า
- weight: น้ำหนักรถยนต์
- acceleration: อัตราเร่ง (mph)
- model_year: ปีที่ผลิต
- origin: แหล่งผลิต (USA, Europe, Japan)
- name: ชื่อรุ่นรถ

Descriptive Statistics ဗဝ်းပုဂံမုာ် mpg

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	count	percent
count	398.000000	398.000000	398.000000	392.000000	398.000000	398.000000	398.000000	europa	70	17.59%
mean	23.514573	5.454774	193.425879	104.469388	2970.424623	15.568090	76.010050	japan	79	19.85%
std	7.815984	1.701004	104.269838	38.491160	846.841774	2.757689	3.697627	usa	249	62.56%
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.000000	70.000000			
25%	17.500000	4.000000	104.250000	75.000000	2223.750000	13.825000	73.000000			
50%	23.000000	4.000000	148.500000	93.500000	2803.500000	15.500000	76.000000			
75%	29.000000	8.000000	262.000000	126.000000	3608.000000	17.175000	79.000000			
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.800000	82.000000			

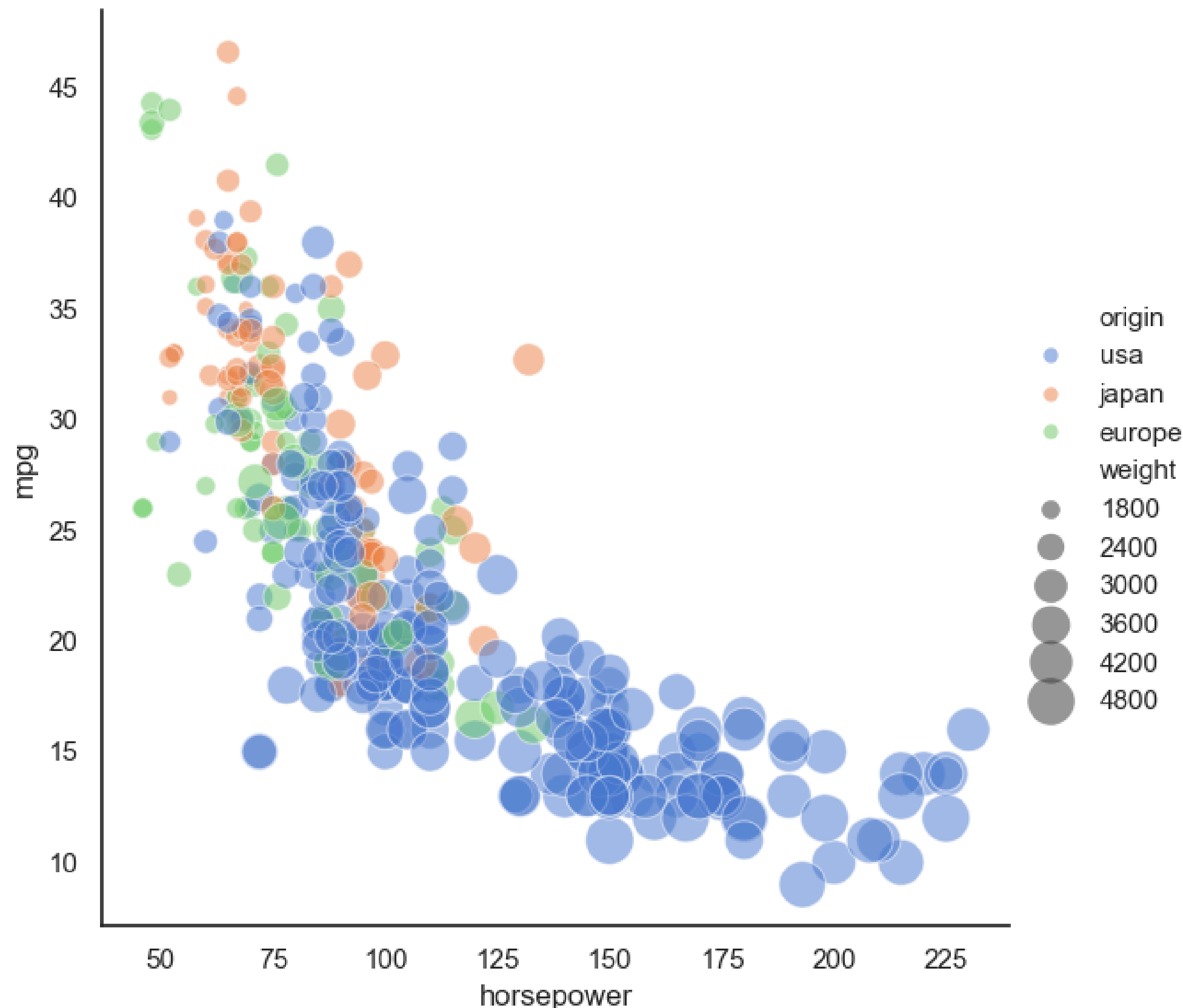
	name	count	percent
0	amc ambassador brougham	1	0.25%
1	amc ambassador dpl	1	0.25%
...	...	...	...
300	vw dasher (diesel)	1	0.25%
301	vw pickup	1	0.25%
302	vw rabbit	2	0.5%
303	vw rabbit c (diesel)	1	0.25%
304	vw rabbit custom	1	0.25%

ไพล์โค้ด สำหรับ Scatter Plot with Marginal Ticks (mpg)

```
1 import seaborn as sns
2 sns.set_theme(style="white", color_codes=True)
3 mpg = sns.load_dataset("mpg")
4
5 # Use JointGrid directly to draw a custom plot
6 g = sns.JointGrid(data=mpg, x="mpg", y="acceleration", space=0, ratio=17)
7 g.plot_joint(sns.scatterplot, size=mpg["horsepower"], sizes=(30, 120),
8 | | | color="g", alpha=.6, legend=True)
9 g.plot_marginals(sns.rugplot, height=1, color="g", alpha=.6)
```

- `sns.JointGrid`: สร้างโครงสร้างกราฟเปล่าที่มีพื้นที่ตรงกลางและพื้นที่ขอบ
 - `space=0`: กำหนดให้ไม่มีช่องว่างระหว่างกราฟหลักและกราฟขอบ
 - `ratio=17`: กำหนดสัดส่วนให้กราฟหลักมีขนาดใหญ่กว่ากราฟขอบ 17 เท่า
- `g.plot_joint`: วาดกราฟหลักที่ตรงกลาง
 - `sns.scatterplot`: ใช้ `scatterplot` เพื่อดูความสัมพันธ์ระหว่าง mpg (แกน X) และ acceleration (แกน Y)
 - `size=mpg["horsepower"]`: กำหนดให้ขนาดของจุดจะแปรผันตาม "แรงม้า"
- `g.plot_marginals`: วาดกราฟที่ขอบด้านบนและด้านขวา
 - `sns.rugplot`: วาดเส้นเล็กๆ เพื่อบอกความหนาแน่นของข้อมูลในแต่ละแกน
 - `height=1`: ให้เส้น Rug ลากยาว 100% เต็มพื้นที่ช่อง Marginal (เทียบกับ `ratio=17`)

5.3.11 Scatterplot with Varying Point Sizes and Hues (mpg)



mpg คือ ชุดข้อมูลประสิทธิภาพการใช้น้ำมันของรถยนต์
มีทั้งหมด 398 แถว 9 คอลัมน์ ได้แก่

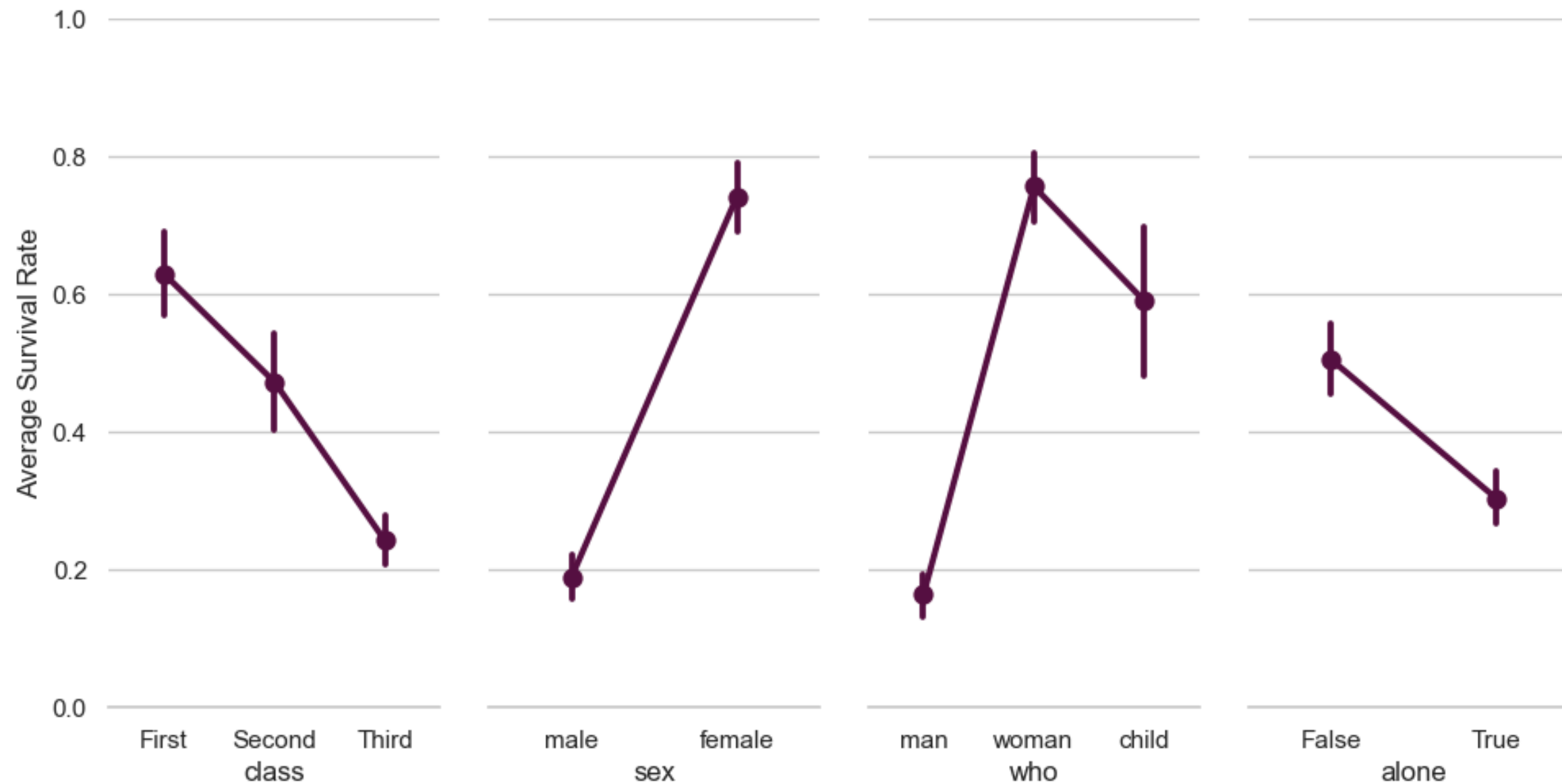
- **mpg**: ระยะทางที่วิ่งได้ ไมล์ ต่อน้ำมัน 1 แกลลอน
- **cylinders**: จำนวนลูกสูบ
- **displacement**: ปริมาตรกระบอกสูบ (ขนาดเครื่องยนต์)
- **horsepower**: กำลังแรงม้า
- **weight**: น้ำหนักรถยนต์
- **acceleration**: อัตราเร่ง (mph)
- **model_year**: ปีที่ผลิต
- **origin**: แหล่งผลิต (USA, Europe, Japan)
- **name**: ชื่อรุ่นรถ

ไพล็อตสำหรับ Scatterplot with Varying Point Sizes and Hues (mpg)

```
1 import seaborn as sns
2 sns.set_theme(style="white")
3
4 # Load the example mpg dataset
5 mpg = sns.load_dataset("mpg")
6
7 # Plot miles per gallon against horsepower with other semantics
8 sns.relplot(x="horsepower", y="mpg", hue="origin", size="weight",
9             sizes=(40, 400), alpha=.5, palette="muted",
10            height=6, data=mpg)
```

- `sns.relplot`: เป็นการสร้าง Relational Plot ซึ่งเป็นรูปแบบหนึ่งของ Scatter Plot ที่ใช้ในการกำหนดขนาด (Size) และ สี (Color) ที่แตกต่างกันได้

5.3.12 Paired Categorical Plots (Titanic)



Titanic คือ ชุดข้อมูลผู้โดยสารบนเรือ Titanic ที่ล่มเมื่อปี 1912 มักใช้เพื่อฝึกการทำ Binary Classification มีทั้งหมด 891 แถว 15 คอลัมน์ โดยมี 5 คอลัมน์หลัก ได้แก่

- survived: สถานะการรอดชีวิต (0 = เสียชีวิต, 1 = รอดชีวิต) เป็นค่าเฉลี่ย
- class: ชั้นที่นั่งของผู้โดยสาร
- sex: เพศ
- who: กลุ่มผู้โดยสาร
- alone: เดินทางคนเดียวหรือไม่

Descriptive Statistics ของชุดข้อมูล Titanic

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

class	count	percent
First	216	24.24%
Second	184	20.65%
Third	491	55.11%

embark_town	count	percent
Cherbourg	168	18.9%
Queenstown	77	8.66%
Southampton	644	72.44%

who	count	percent
child	83	9.32%
man	537	60.27%
woman	271	30.42%

alive	count	percent
no	549	61.62%
yes	342	38.38%

adult_male	count	percent
False	354	39.73%
True	537	60.27%

alone	count	percent
False	354	39.73%
True	537	60.27%

sex	count	percent
female	314	35.24%
male	577	64.76%

embarked	count	percent
C	168	18.9%
Q	77	8.66%
S	644	72.44%

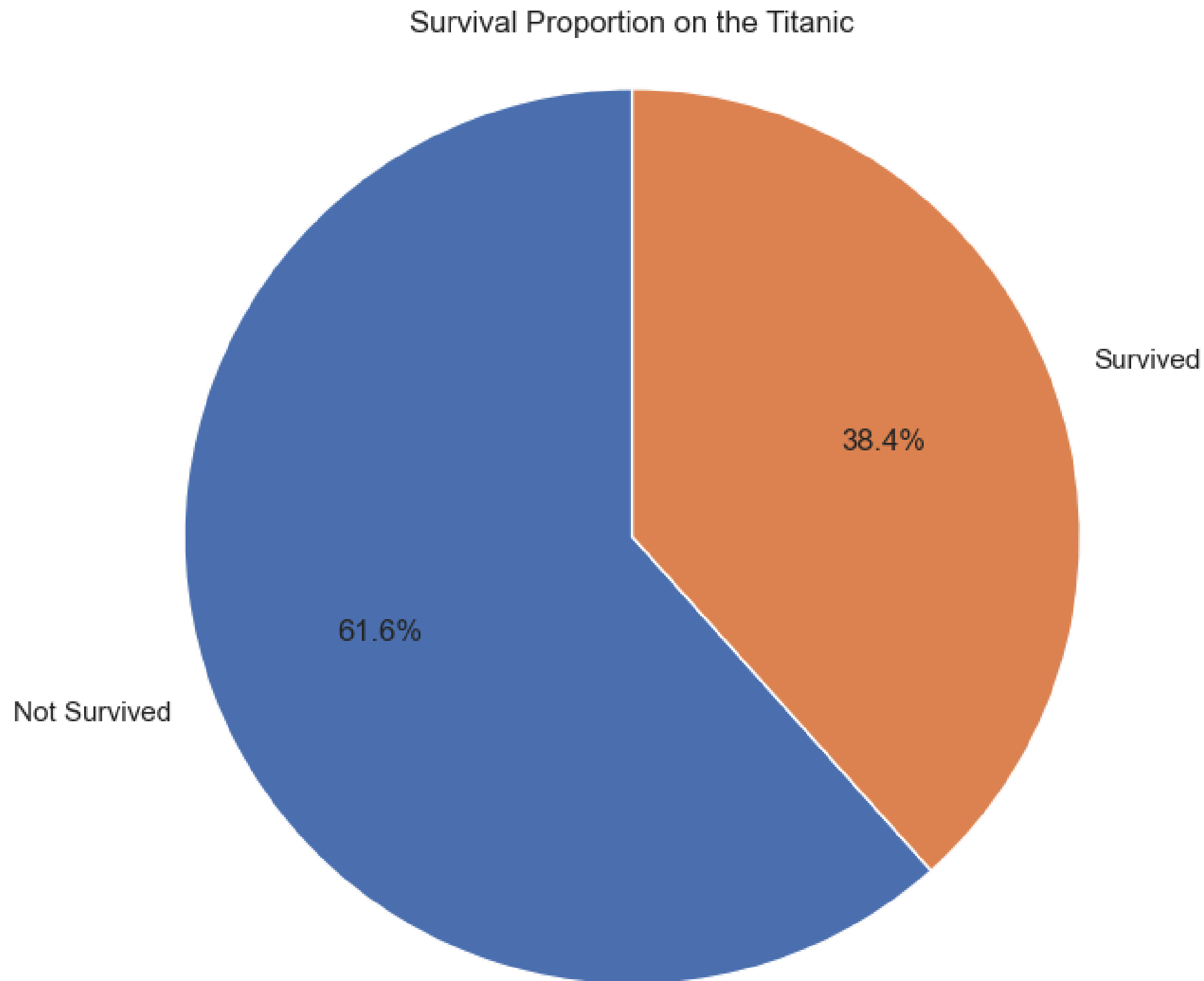
deck	count	percent
A	15	7.39%
B	47	23.15%
C	59	29.06%
D	33	16.26%
E	32	15.76%
F	13	6.4%
G	4	1.97%

ไพลทอนโค้ด สำหรับ Paired Categorical Plots (Titanic)

```
1 import seaborn as sns
2 sns.set_theme(style="whitegrid")
3
4 # Load the example Titanic dataset
5 titanic = sns.load_dataset("titanic")
6
7 # Set up a grid to plot survival probability against several variables
8 g = sns.PairGrid(titanic, y_vars="survived",
9                  x_vars=["class", "sex", "who", "alone"],
10                  height=5, aspect=.5)
11
12 # Draw a seaborn pointplot onto each Axes
13 g.map(sns.pointplot, color="xkcd:plum")
14 g.set(ylim=(0, 1))
15 g.axes[0, 0].set_ylabel("Average Survival Rate")
16 sns.despine(fig=g.fig, left=True)
```

- `sns.PairGrid`: สร้างตารางกราฟเปล่า ๆ ขึ้นมา
 - `y_vars="survived"`: กำหนดให้แกน Y ของทุกกราฟคืออัตราการรอดชีวิต
 - `x_vars=["class", "sex", "who", "alone"]`: กำหนดตัวแปรต้นที่ต้องการเปรียบเทียบในแต่ละคอลัมน์
 - `height=5, aspect=.5`: กำหนดให้แต่ละกราฟย่อยสูง 5 นิ้ว กว้าง 0.5 เท่าของ 5 นิ้ว
- `g.map(sns.pointplot, ...)`: สั่งให้วาด Pointplot ลงในทุกช่องของ Grid
 - Pointplot: จุดวงกลมคือ "ค่าเฉลี่ย" (Mean) ของอัตราการรอดชีวิต ส่วนเส้นแนวตั้งคือช่วงความเชื่อมั่น
 - `color="xkcd:plum"`: ใช้สีม่วงพลัมตามมาตรฐานชื่อสีของ XKCD
- `g.set(ylim=(0, 1))`: กำหนดให้แกน Y มีค่าตั้งแต่ 0 ถึง 1 (0% ถึง 100%) เพื่อให้เห็นสัดส่วนความน่าจะเป็นที่ชัดเจน
- `sns.despine(fig=g.fig, left=True)`: เอาเส้นขอบด้านซ้ายออกเพื่อให้กราฟดูโปร่งและทันสมัย

5.3.13 Pie Chart (Titanic)



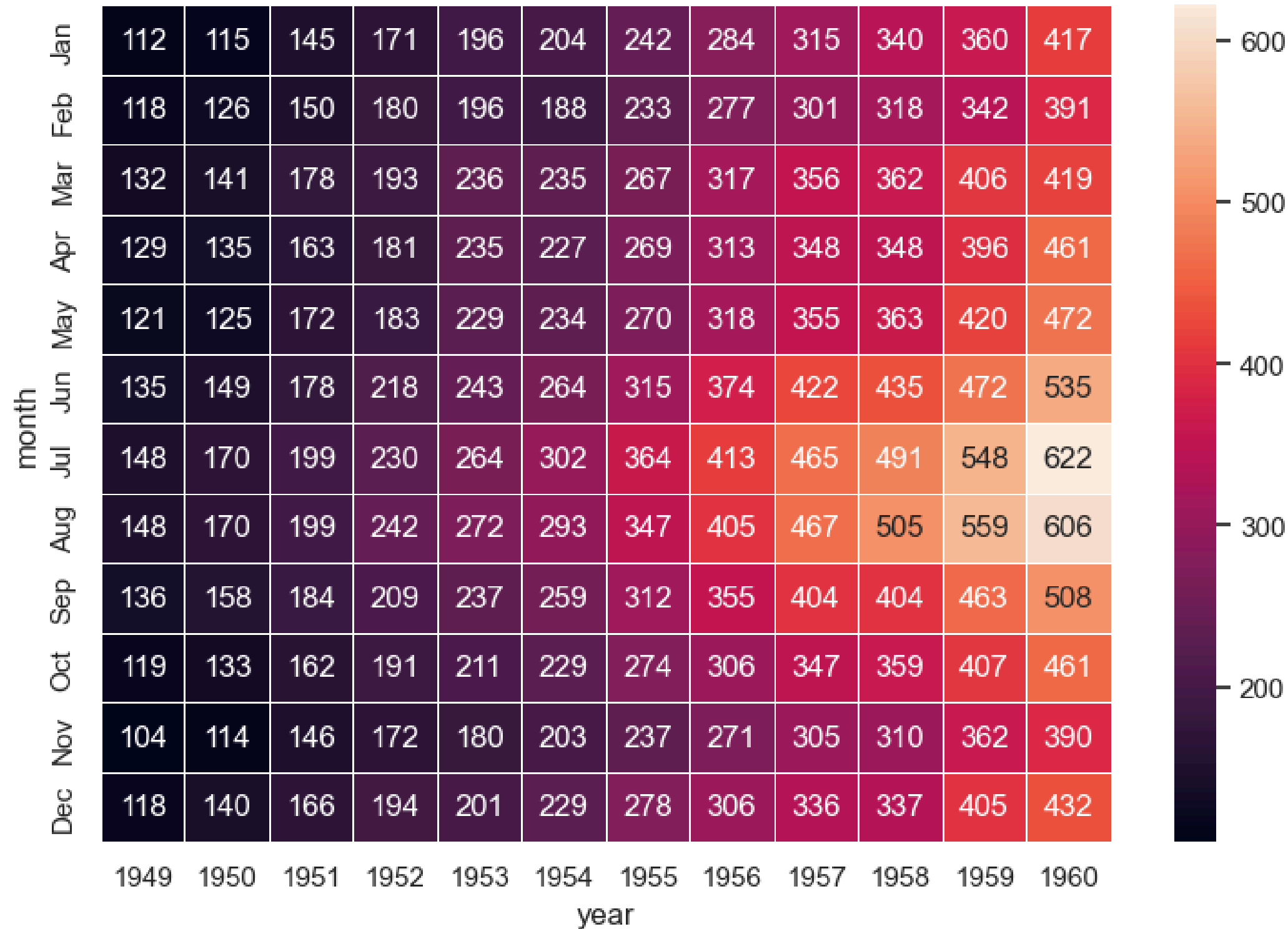
Titanic คือ ชุดข้อมูลผู้โดยสารบนเรือ Titanic ที่ล่มเมื่อปี 1912 มักใช้เพื่อฝึกการทำ Binary Classification มีทั้งหมด 891 แถว 15 คอลัมน์ โดยมี 5 คอลัมน์หลัก ได้แก่

- survived: สถานะการรอดชีวิต (0 = เสียชีวิต, 1 = รอดชีวิต) เป็นค่าเฉลี่ย
- class: ชั้นที่นั่งของผู้โดยสาร
- sex: เพศ
- who: กลุ่มผู้โดยสาร
- alone: เดินทางคนเดียวหรือไม่

ไพล็อตสำหรับ Pie Chart (Titanic)

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Load dataset
5 titanic = sns.load_dataset("titanic")
6
7 # Prepare data: count survived vs not survived
8 survival_counts = titanic["survived"].value_counts().sort_index()
9
10 # Create pie chart
11 plt.figure(figsize=(7, 7))
12 plt.pie(
13     survival_counts,
14     labels=["Not Survived", "Survived"],
15     autopct="%.1f%%",
16     startangle=90
17 )
18
19 plt.title("Survival Proportion on the Titanic")
20 plt.axis("equal") # ให้ pie เป็นวงกลม
21 plt.show()
```

5.3.14 Annotated Heatmaps (Flights)



Flights คือ ชุดข้อมูลที่แสดงจำนวนผู้โดยสารของสายการบิน มีทั้งหมด 144 แถว 3 คอลัมน์ ได้แก่

- year: ปี ค.ศ. (ตั้งแต่ 1949, 1950, ..., 1960)
- month: ชื่อเดือน (January - December)
- passengers: จำนวนผู้โดยสารในเดือนนั้น ๆ

Descriptive Statistics **သံသုတေသန** Flights

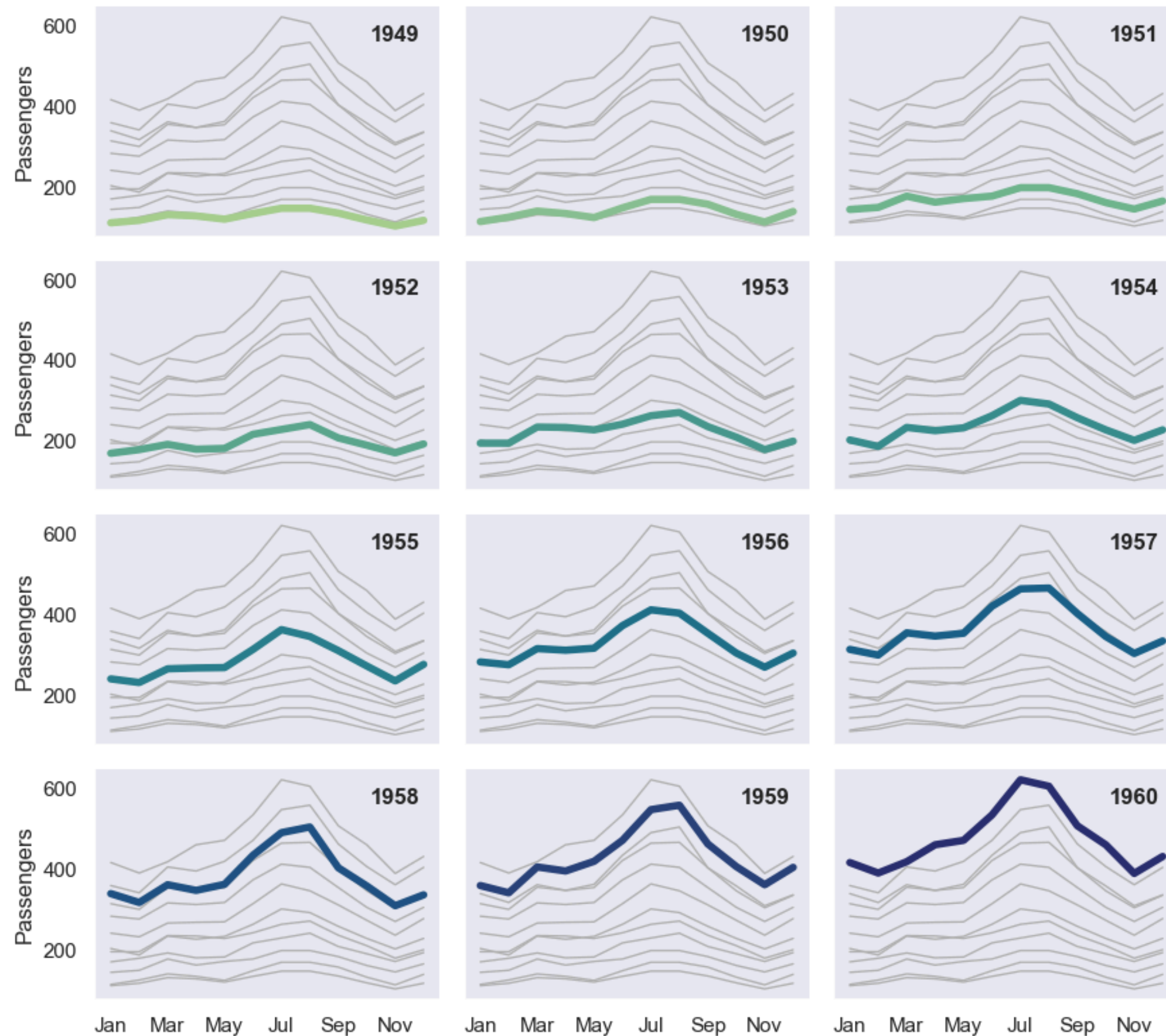
	year	passengers	month	count	percent
count	144.000000	144.000000	Jan	12	8.33%
mean	1954.500000	280.298611	Feb	12	8.33%
std	3.464102	119.966317	Mar	12	8.33%
min	1949.000000	104.000000	Apr	12	8.33%
25%	1951.750000	180.000000	May	12	8.33%
50%	1954.500000	265.500000	Jun	12	8.33%
75%	1957.250000	360.500000	Jul	12	8.33%
max	1960.000000	622.000000	Aug	12	8.33%
			Sep	12	8.33%
			Oct	12	8.33%
			Nov	12	8.33%
			Dec	12	8.33%

ไพลทโค้ด สำหรับ Annotated Heatmaps (Flights)

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 sns.set_theme()
4
5 # Load the example flights dataset and convert to long-form
6 flights_long = sns.load_dataset("flights")
7 flights = (
8     flights_long
9     .pivot(index="month", columns="year", values="passengers")
10 )
11
12 # Draw a heatmap with the numeric values in each cell
13 f, ax = plt.subplots(figsize=(9, 6))
14 sns.heatmap(flights, annot=True, fmt="d", linewidths=.5, ax=ax)
```

- `.pivot(index="month", columns="year", values="passengers")`: แปลงข้อมูลจากรูปแบบ List ยาว ๆ ให้เป็น ตาราง Matrix โดย:
 - แกว (Index) คือ เดือน (มกราคม - ธันวาคม)
 - คอลัมน์ (Columns) คือ ปี ค.ศ.
 - ค่าในช่อง (Values) คือ จำนวนผู้โดยสาร
- `sns.heatmap`: เป็นการสร้าง Heatmaps เพื่อแสดงความหนาแน่นของข้อมูลเชิงปริมาณในรูปแบบตาราง
 - `annot=True`: ย่อมาจาก annotation คือการให้ แสดงตัวเลข ลงไปในแต่ละช่องของตาราง
 - `fmt="d"`: ย่อมาจาก decimal integer เพื่อบอกให้แสดงตัวเลขในรูปแบบ จำนวนเต็ม
 - `linewidths=.5`: กำหนดขนาดของเส้นระหว่างช่อง
 - `ax=ax`: ระบุว่าจะให้วาดกราฟลงในพื้นที่ (Axes) ที่ได้เตรียมไว้

5.3.15 Small Multiple Time Series (Flights)



Flights คือ ชุดข้อมูลที่แสดงจำนวนผู้โดยสารของสายการบิน มีทั้งหมด 144 แถว 3 คอลัมน์ ได้แก่

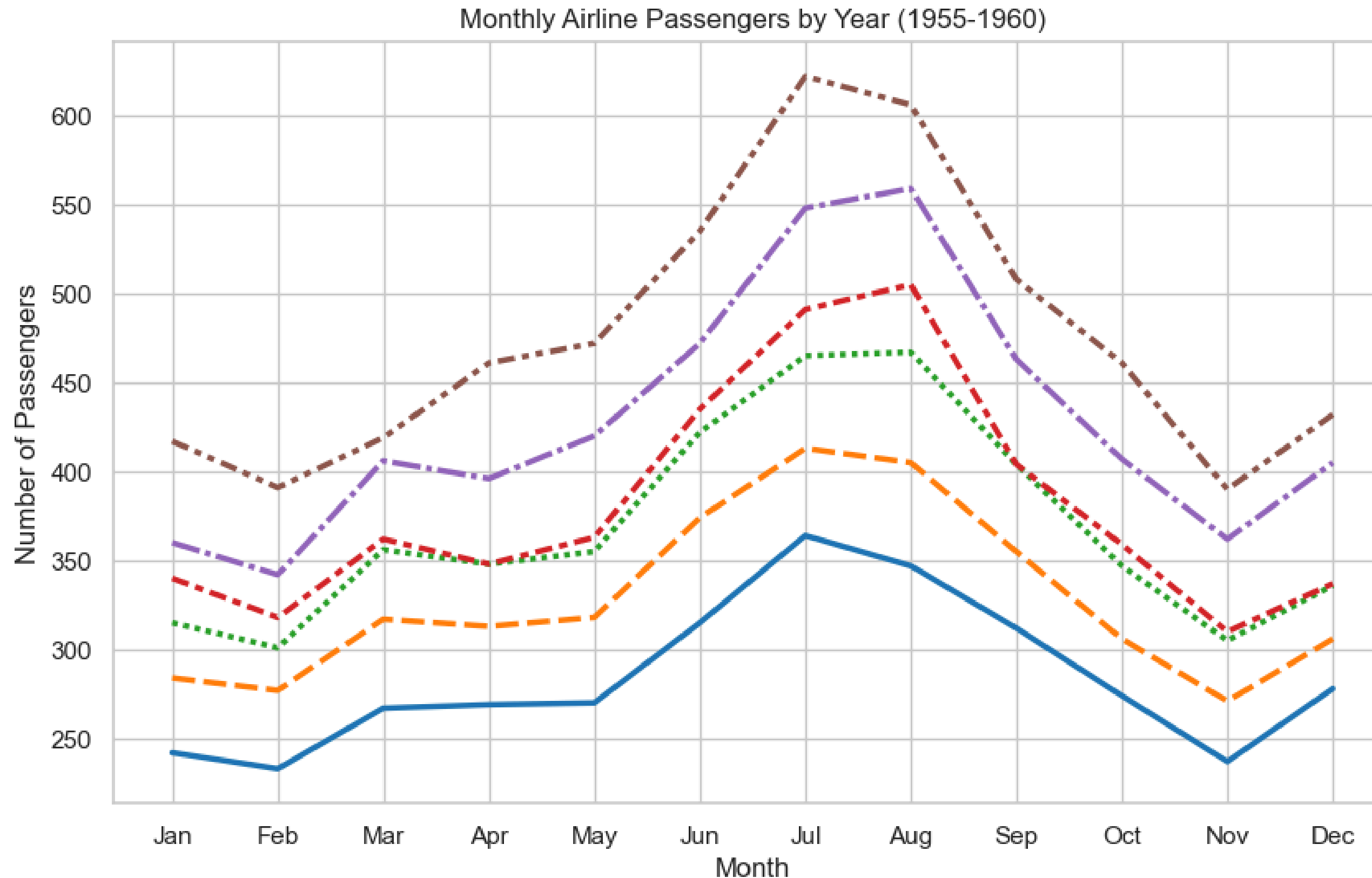
- year: ปี ค.ศ. (ตั้งแต่ 1949, 1950, ..., 1960)
- month: ชื่อเดือน (January - December)
- passengers: จำนวนผู้โดยสารในเดือนนั้น ๆ

ไพล็อตสำหรับ Small Multiple Time Series (Flights)

```
1 import seaborn as sns
2
3 sns.set_theme(style="dark")
4 flights = sns.load_dataset("flights")
5
6 # Plot each year's time series in its own facet
7 g = sns.relplot(
8     data=flights,
9     x="month", y="passengers", col="year", hue="year",
10    kind="line", palette="crest", linewidth=4, zorder=5,
11    col_wrap=3, height=2, aspect=1.5, legend=False,
12 )
13
14 # Iterate over each subplot to customize further
15 for year, ax in g.axes_dict.items():
16
17     # Add the title as an annotation within the plot
18     ax.text(.8, .85, year, transform=ax.transAxes, fontweight="bold")
19
20     # Plot every year's time series in the background
21     sns.lineplot(
22         data=flights, x="month", y="passengers", units="year",
23         estimator=None, color=".7", linewidth=1, ax=ax,
24     )
25
26 # Reduce the frequency of the x axis ticks
27 ax.set_xticks(ax.get_xticks()[::2])
28
29 # Tweak the supporting aspects of the plot
30 g.set_titles("")
31 g.set_axis_labels("", "Passengers")
32 g.tight_layout()
```

- `sns.relplot`: สร้าง Relational Plot และกำหนดให้เป็นกราฟหลัก
 - `col="year"`: แยกกราฟออกเป็นช่องๆ (Facets) ตามปี (1 ปีต่อ 1 กรอบ)
 - `col_wrap=3`: กำหนดให้แสดงเพียง 3 กรอบต่อหนึ่งแถว (ถ้ามี 12 ปี จะได้ 4 แถว)
 - `kind="line"`: ระบุว่าเป็นกราฟเส้น
 - `palette="crest"`: ใช้โทนสีไล่ระดับจากน้ำเงินไปเขียวตามปีที่เพิ่มขึ้น
 - `linewidth=4`: กำหนดความหนาของเส้นเป็น 4 หน่วย
 - `zorder=5`: กำหนดเส้นในกราฟนี้ให้อยู่ชั้นที่ 5 หรือชั้นบน ๆ เพื่อป้องกันไม่ให้โดนเส้นจากกราฟอื่นทับ
- `g.axes_dict.items()`: วนลูปเข้าไปจัดการในแต่ละกรอบกราฟ
- `ax.text(...)`: เขียนตัวเลขปีลงไปในพื้นที่กราฟ (แทนการใช้ Title แบบปกติ)
- `sns.lineplot`: วาดเส้นของ "ทุกปี" ลงไปในทุกกรอบ แต่ใช้สีเทาจาง ๆ และเส้นบาง เพื่อใช้เป็นเงาเปรียบเทียบกับกราฟหลัก
 - `estimator=None`: ไม่ต้องคำนวณค่าทางสถิติใดๆ
 - `color=".7"`: กำหนดสีเทาจาง ๆ
 - `linewidth=1`: กำหนดความหนาของเส้นเป็น 1 หน่วย

5.3.16 Multi-Series Line Chart (Flights)



Flights คือ ชุดข้อมูลที่แสดงจำนวนผู้โดยสารของสายการบิน มีทั้งหมด 144 แถว 3 คอลัมน์ ได้แก่

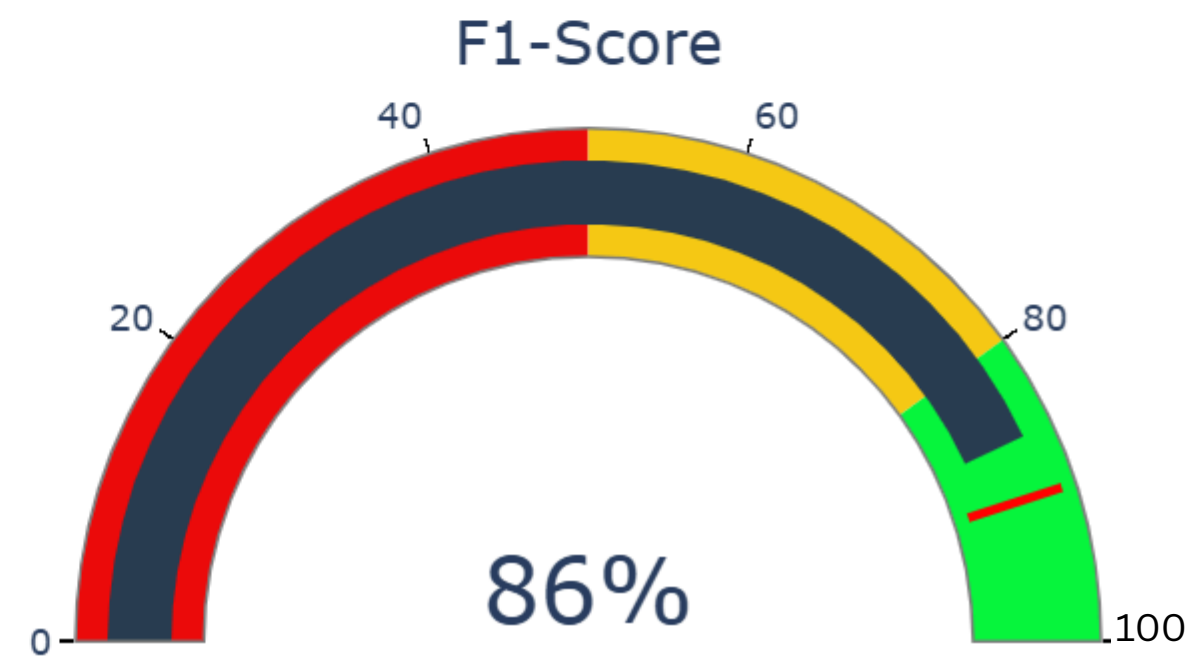
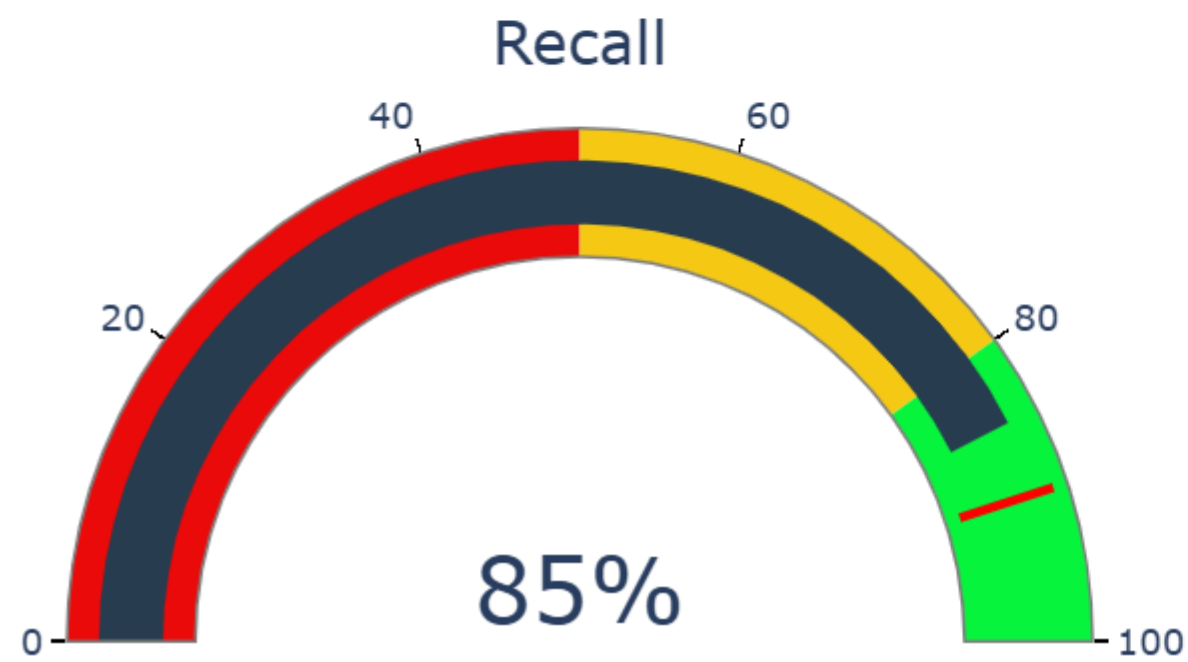
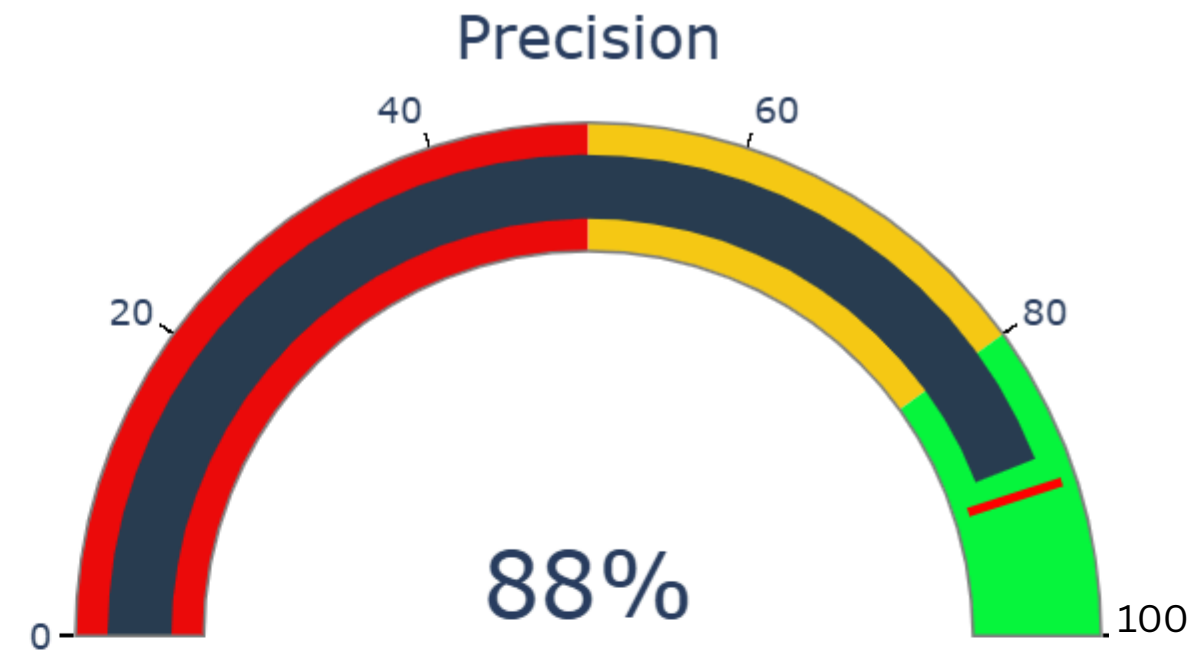
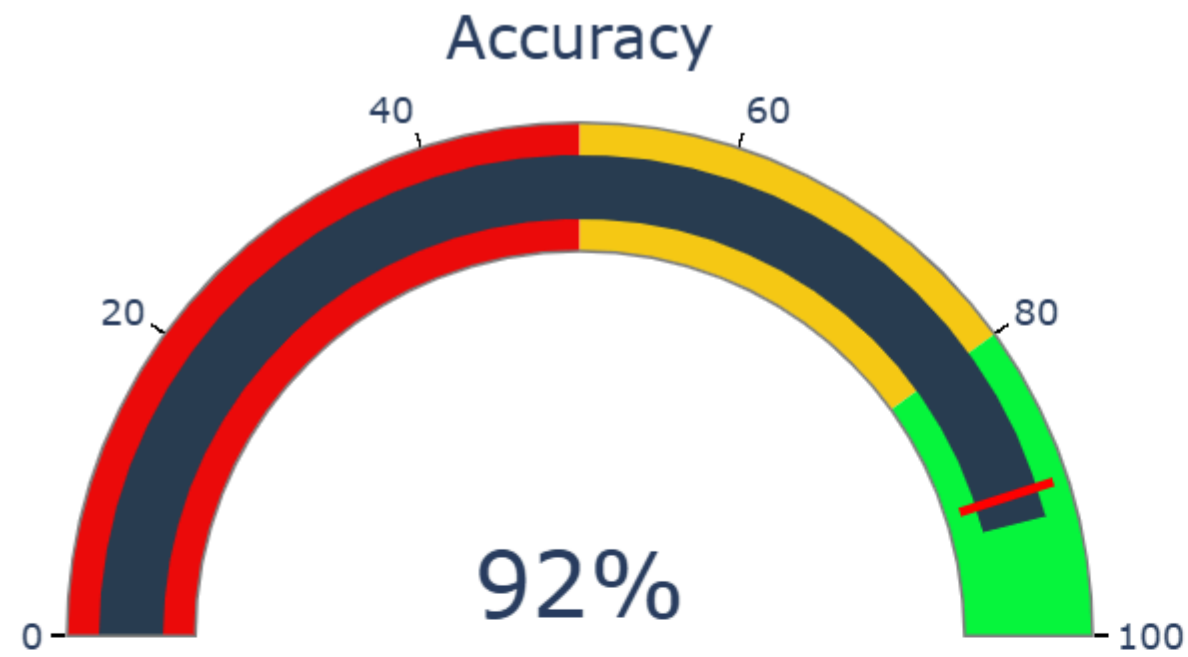
- year: ปี ค.ศ.
- month: ชื่อเดือน
- passengers: จำนวนผู้โดยสาร

ไพล็อตสำหรับ Multi-Series Line Chart (Flights)

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.set_theme(style="whitegrid")
5 flights = sns.load_dataset("flights")
6
7 years_to_show = [1955, 1956, 1957, 1958, 1959, 1960]
8 data_filtered = flights[flights['year'].isin(years_to_show)]
9 data_plot = data_filtered.pivot(index="month", columns="year", values="passengers")
10
11 plt.figure(figsize=(10, 6))
12 sns.lineplot(data=data_plot, palette="tab10", linewidth=2.5)
13
14 plt.title("Monthly Airline Passengers by Year (1955-1960)")
15 plt.ylabel("Number of Passengers")
16 plt.xlabel("Month")
17 plt.legend(title="Year", bbox_to_anchor=(1.05, 1), loc='upper left')
18 plt.show()
```

- sns.lineplot: สร้าง Line Plot
 - palette="tab10": เลือกชุดสีมาตรฐาน 10 สีที่ตัดกันชัดเจน ช่วยให้แยกแต่ละปีได้ง่าย
 - linewidth=2.5: กำหนดความหนาของเส้นเป็น 2.5 หน่วย

5.3.17 Gauge Chart



ไพล็อตสำหรับ Gauge Chart

```
1 import plotly.graph_objects as go
2 from plotly.subplots import make_subplots
3
4 # สมมติค่า Metrics (ค่าจริงมักได้จาก sklearn.metrics)
5 metrics = {
6     "Accuracy": 0.92,
7     "Precision": 0.88,
8     "Recall": 0.85,
9     "F1-Score": 0.86
10 }
11
12 # สร้าง Layout แบบ 2 แถว 2 คอลัมน์
13 fig = make_subplots(
14     rows=2, cols=2,
15     specs=[[{'type': 'indicator'}, {'type': 'indicator'}],
16           [{'type': 'indicator'}, {'type': 'indicator'}]],
17     vertical_spacing=0.2
18 )
19
```

```

20 # วนลูปสร้าง Gauge สำหรับแต่ละ Metric
21 for i, (name, val) in enumerate(metrics.items()):
22     row = (i // 2) + 1
23     col = (i % 2) + 1
24
25     fig.add_trace(go.Indicator(
26         mode = "gauge+number",
27         value = val * 100,
28         number = {'suffix': "%", 'font': {'size': 30}},
29         title = {'text': name, 'font': {'size': 20}},
30         gauge = {
31             'axis': {'range': [0, 100], 'tickwidth': 1, 'tickcolor': "black"},
32             'bar': {'color': "#2c3e50"}, # สีเข้ม/แถบพลัง
33             'bgcolor': "white",
34             'borderwidth': 1,
35             'bordercolor': "gray",
36             'steps': [
37                 {'range': [0, 50], 'color': "#ed0a0a"}, # แดงอ่อน
38                 {'range': [50, 80], 'color': "#f5cc17"}, # เหลืองอ่อน
39                 {'range': [80, 100], 'color': "#06f73e"} # เขียวอ่อน
40             ],
41             'threshold': {
42                 'line': {'color': "red", 'width': 3},

```

```
43         'thickness': 0.75,
44         'value': 90}
45     }
46     ), row=row, col=col)
47
48 # ปรับแต่ง Layout โดยรวมให้เป็นพื้นขาว
49 fig.update_layout(
50     paper_bgcolor = "white",
51     plot_bgcolor = "white",
52     margin = dict(l=20, r=20, t=50, b=20),
53     height=600,
54     width=800
55 )
56
57 fig.show()
```

5.3.18 Candlestick



ไพล็อตสำหรับ Candlestick (1)

```
1 import plotly.graph_objects as go
2 import yfinance as yf
3 df = yf.download("AAPL", start="2021-02-27", end="2026-02-27", progress=False)
4 #df = yf.download("AAPL", period="4mo", interval="1d")
5 df.columns = df.columns.droplevel(1) # แก่ MultiIndex columns
6 df = df[['Open', 'High', 'Low', 'Close', 'Volume']]
7 df = df.sort_index() # เรียงวันที่จากเก่า → ใหม่
8 #df.to_excel("AAPL.xlsx")# Export to Excel
9
10 fig = go.Figure()
11
12 fig.add_trace(
13     go.Candlestick(
14         x=df.index,
15         open=df['Open'],
16         high=df['High'],
17         low=df['Low'],
18         close=df['Close'])
19 )
20 fig.update_layout(
21     margin=dict(l=10, r=10, t=40, b=10),
22     width=950,      # ความกว้าง
23     height=600      # ความสูง
24 )
25 fig.show()
```

✓ 0.0s

Python



ไพล็อตสำหรับ Candlestick (2)

```
1 import mplfinance as mpf
2
3 mpf.plot(
4     data=df[df.index > '2025-11-27'],
5     type='candle',          # candlestick
6     volume=True,           # แสดง volume
7     style='yahoo',         # theme
8     title='AAPL Stock Price',
9     ylabel='Price (USD)',
10    ylabel_lower='Volume',
11    mav=(7, 25),            # Moving Average
12    figsize=(12, 8),
13    tight_layout=True,      # ลดขอบขาวส่วนเกิน
14 )
15
```

✓ 0.5s

Python



ไพทอนโค้ด สำหรับ Candlestick (3)

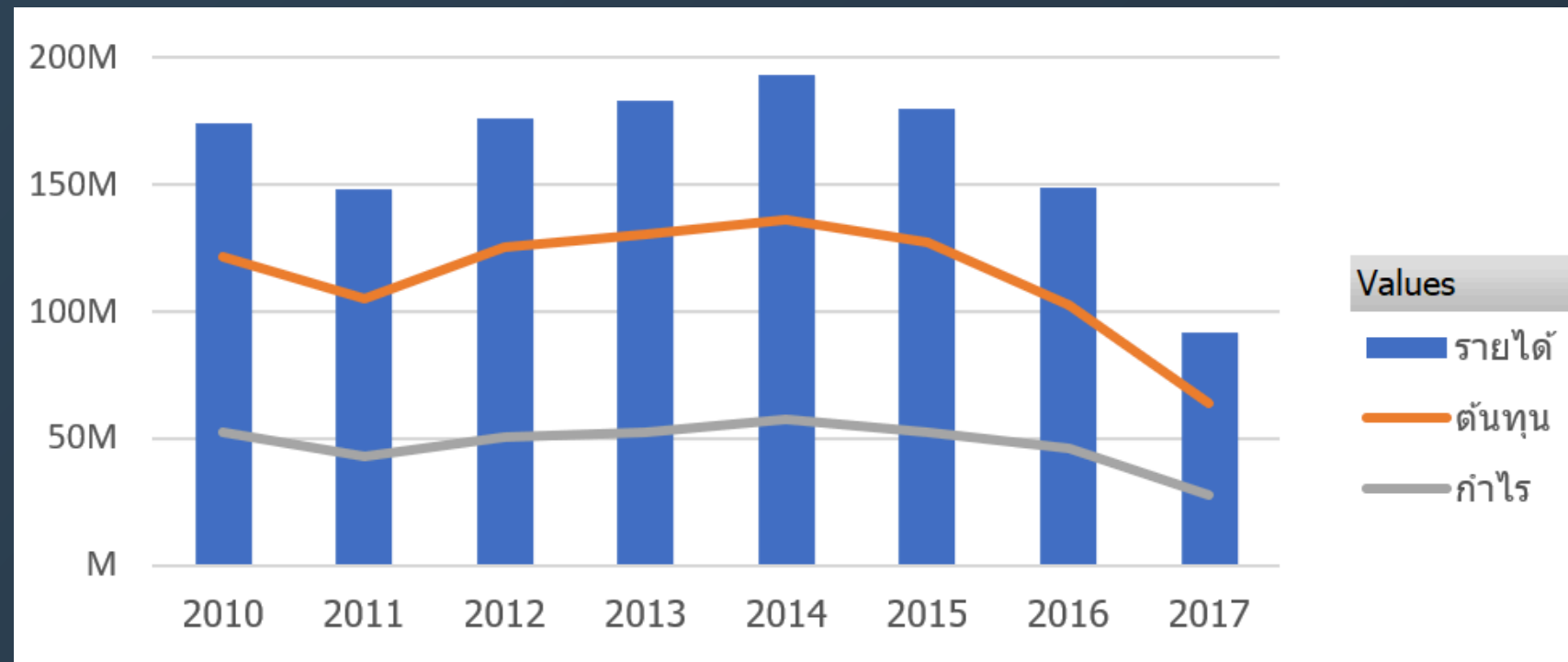
```
1 import mplfinance as mpf
2
3 # ปรับแต่ง Style ให้เน้นความชัดเจนของ Body แท่งเทียน
4 colors = mpf.make_marketcolors(
5     up='█ '#26a69a',          # เขียวสว่าง
6     down='█ '#ef5350',        # แดงสว่าง
7     edge='black',             # ใส่ขอบสีดำให้เห็นขอบเขตแท่งชัดเจน
8     wick='black',             # ใส่เทียนสีดำเข้ม
9     volume='in'               # ให้แท่ง Volume ใช้สีเดียวกับแท่งเทียน
10 )
11
12 custom_style = mpf.make_mpf_style(
13     marketcolors=colors,
14     gridstyle=':',
15     facecolor='█ '#f8f9fa'    # พื้นหลังสีเทาอ่อน สบายตา
16 )
17
18 mpf.plot(
19     data=df[df.index > '2025-11-27'],
20     type='candle',            # candlestick
21     volume=True,              # แสดง volume
22     style=custom_style,       # theme
23     title='AAPL Stock Price',
24     ylabel='Price (USD)',
25     ylabel_lower='Volume',
26     mav=(7, 25),              # Moving Average
27     figsize=(12, 8),
28     tight_layout=True,        # ลดขอบขาวส่วนเกิน
29
30     show_nontrading=False,    # ไม่แสดงช่วงวันที่ไม่มีการซื้อขาย
31     update_width_config=dict(candle_linewidth=1.2, candle_width=0.7) # ปรับให้แท่งอ้วนขึ้น
32 )
```

✓ 0.1s

Python

5.4 การใช้งาน Excel

product_sale.xlsx



ปี	รายได้	ต้นทุน	กำไร
2010	174M	122M	53M
2011	148M	105M	43M
2012	176M	126M	50M
2013	183M	130M	52M
2014	194M	136M	57M
2015	180M	127M	53M
2016	149M	103M	46M
2017	92M	64M	28M
Grand Total	1,296M	914M	382M

	Units Sold	Unit Price	Unit Cost	Incomes	Costs	Profits
Min	13	9.33	6.92	2043.25	1416.75	532.61
Max	9998	668.27	524.96	6617209.54	5204978.4	1726181.36
Mean	5060.37	262.16	185.04	1323768.48	933147.97	390620.52
Median	5251	154.06	97.44	761165.2	465709.13	278084.01
SD	2899.79	216.12	175.28	1479290.47	1157250.1	382562.86
Skewness	-0.06	0.79	0.94	1.63	1.8	1.4
Kurtosis	-1.22	-0.74	-0.6	2.06	2.59	1.61

5.5 การใช้งาน Power BI

- Power BI Desktop

ดาวน์โหลดได้ที่ : <https://apps.microsoft.com/detail/9ntxr16hnw1t?hl=en-US&gl=US>



Power BI Desktop

Microsoft Corporation

4.8 ★ | 12.9K ratings | Business

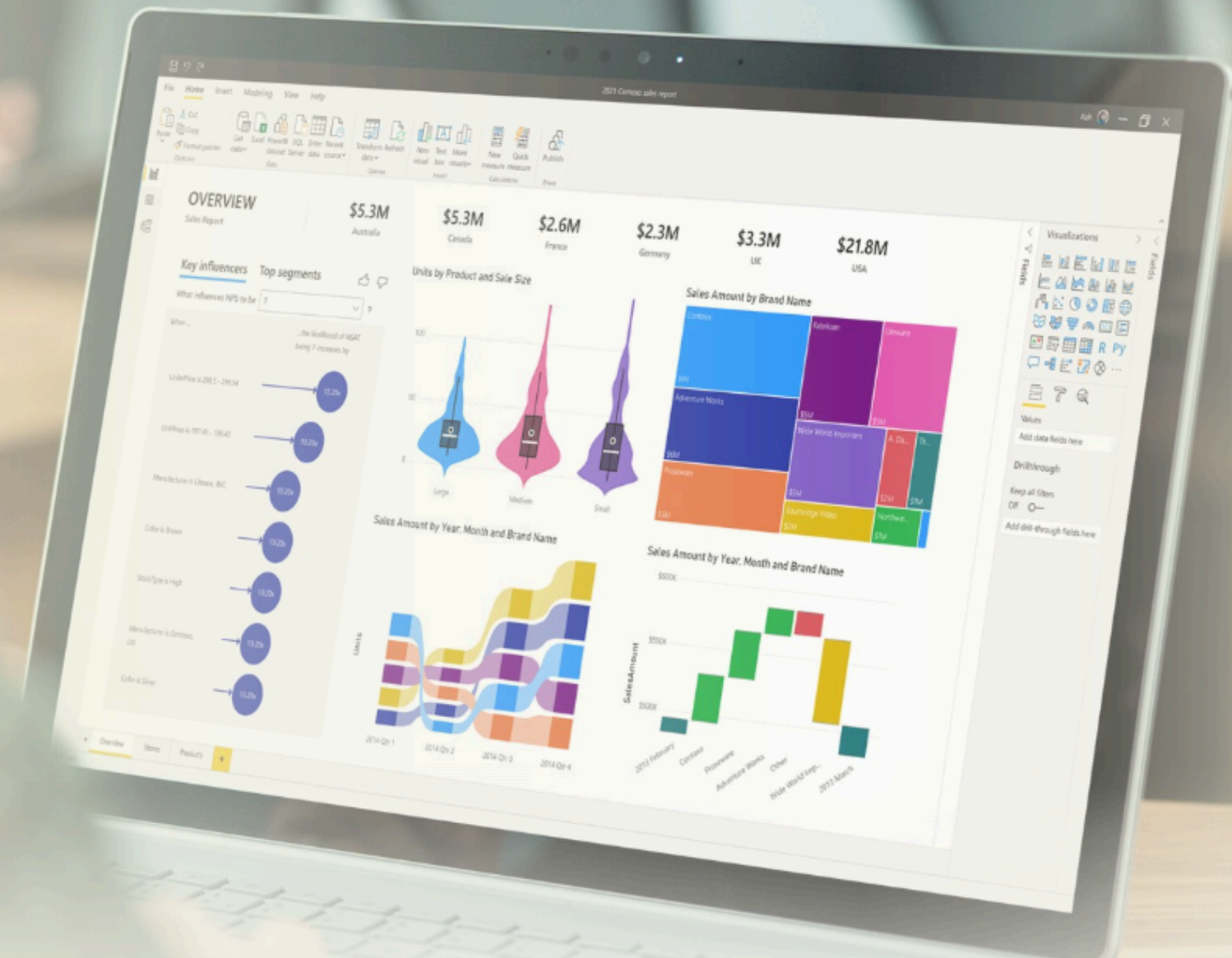
Power BI Desktop puts visual analytics at your fingertips. With this powerful authoring tool, you can create interactive data...

Download

 View in Store

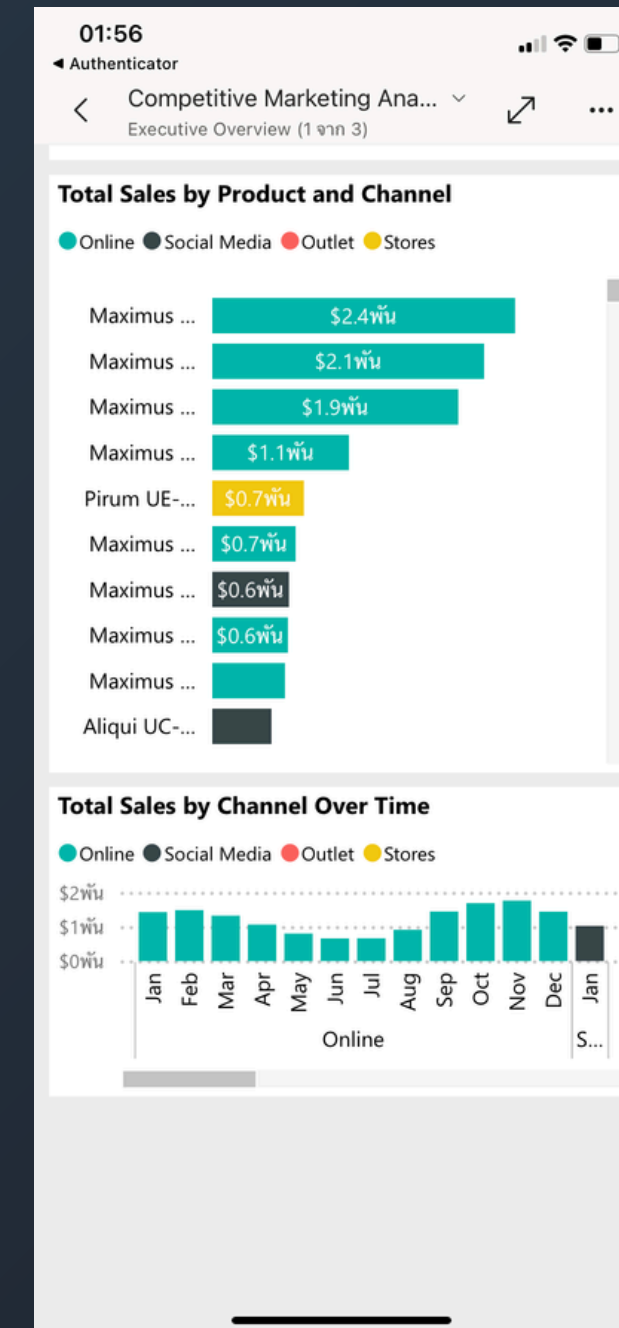
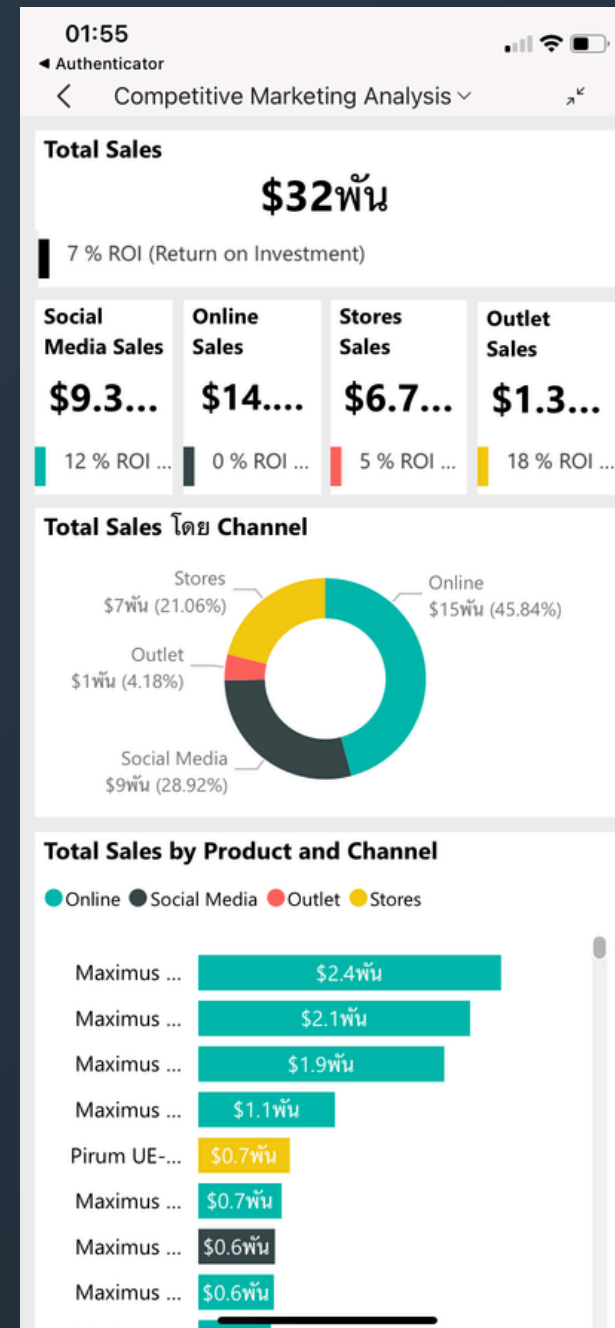
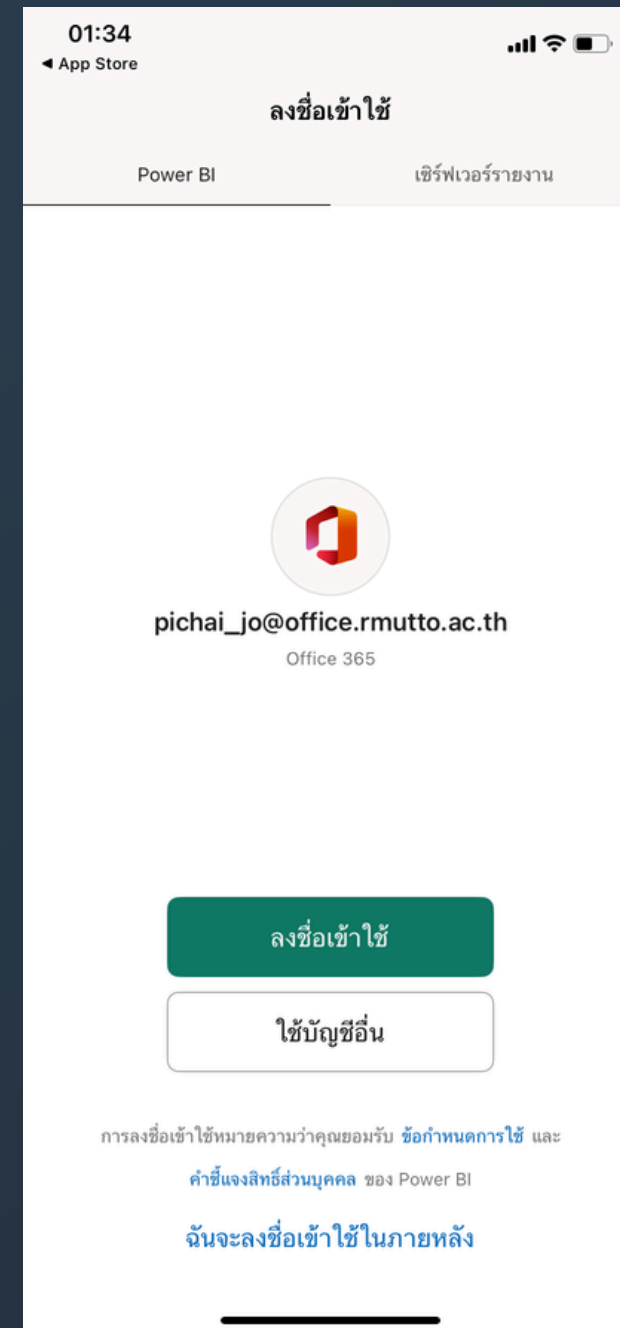
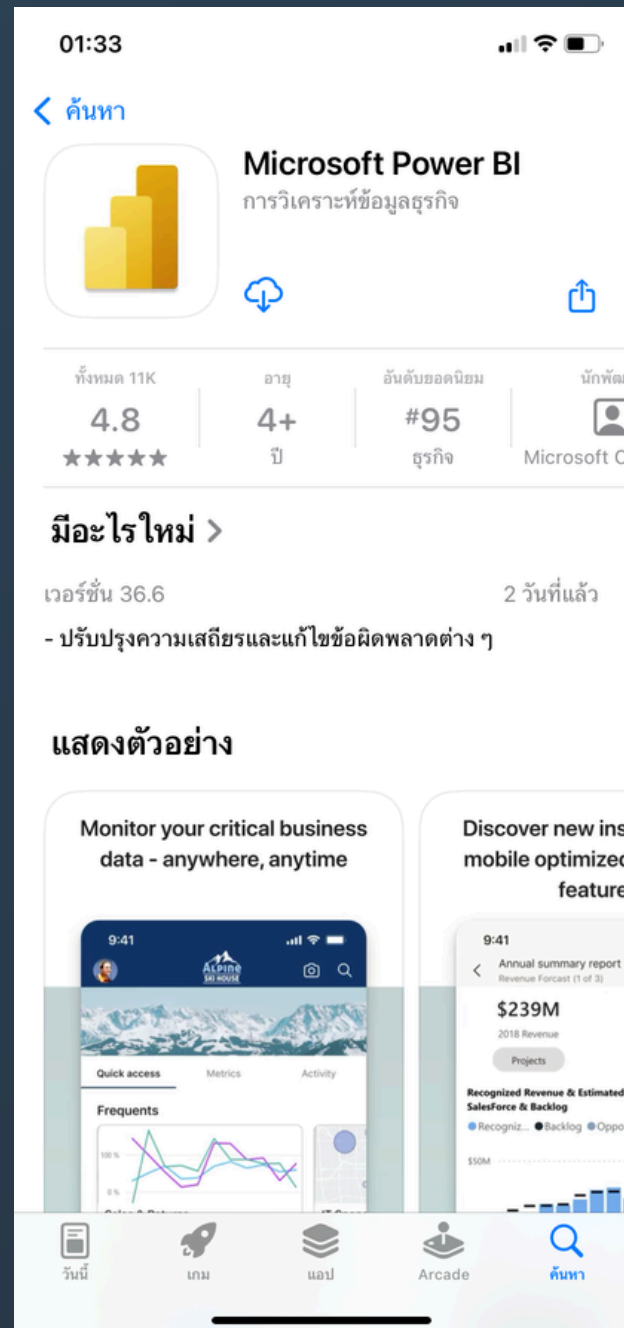


EVERYONE



• Power BI Mobile

ดาวน์โหลดได้ที่ :



• Power BI Web

ใช้งานได้ที่ : <https://app.powerbi.com/>

The screenshot displays the Power BI Web interface. At the top, there's a navigation bar with the Power BI logo, a search bar, and various utility icons. The left sidebar contains navigation links: Home, Copilot, Create, Browse, Workspaces, My workspace, and a specific workspace named 'bi_sale'. The main content area is titled 'Recommended' and features four data story cards. The first card, 'Cancer statistics in the USA', includes a map of the United States and a summary of cancer statistics. The second card, 'Intro—What is Power BI?', provides a guide for getting started. The third card, 'Sports ranked by degree of difficulty, choose...', shows a list of sports ranked by difficulty. The fourth card, 'How equa', discusses gender equality. Each card has an 'Open' button. At the bottom, there are filters for 'Recent', 'Favorites', 'My apps', and 'From external orgs', along with a 'Filter by keyword' search bar.

Power BI Home

Search

New items saved to: My workspace

+ New report

Recommended

Cancer statistics in the USA
Explore this data story

Cancer Analytics Dashboard

TOTAL CANCER INCIDENCES & MORTALITIES

AT A GLANCE... USA CANCER STATISTICS

TOTAL POPULATION AFFECTED: 743M

MALE: 191M, FEMALE: 181M

DEMOGRAPHY: AREA

MAJOR AGE GROUP: 70-74 Years

INCIDENTS: 41.0%, MORTALITY: 10.5%

INFANTS IMPACTED: <1 Year

Open

Intro—What is Power BI?
Getting started with Power BI

Open

Sports ranked by degree of difficulty, choose...
Explore this data story

Ranking Sports by Degree of Difficulty for Key Skills

AGILITY: The ability to change direction quickly.

ANALYTIC APTITUDE: The ability to evaluate and react appropriately to strategic situations.

DURABILITY: The ability to withstand physical punishment over a long period of time.

ENDURANCE: The ability to continue to perform a skill or action for long periods of time.

FLEXIBILITY: The ability to stretch the joints across a large range of motion.

HAND-EYE COORDINATION: The ability to react quickly to sensory perception.

NERVE: The ability to overcome fear.

POWER: The ability to produce strength in the shortest possible time.

SPEED: The ability to move quickly.

Overall Degree of Difficulty (All Skills Combined)

Sport	Overall Degree of Difficulty (All Skills Combined)
Bowling	72.41
Ice Hockey	71.37
Football	68.61
Basketball	67.83
Wrestling	63.3
Martial Arts	63.49
Tennis	62.77
Gymnastics	62.53
Baseball/Softball	62.37
Soccer	61.61
Skiing	60.56
Water Polo	60.06
Rugby	58.33
Lacrosse	58.41
Rodeo: Steer Wrestling	54.33
Track and Field: Pole Vault	54.90
Field Hockey	54.08
Speed Skating	54.90
Figure Skating	54.38
Cycling: Distance	54.41
Volleyball	54.64
Racquetball/Squash	53.65
Surfing	53.35

Explore more... Compare Sports Most Difficult By Skill Data Table

Open

How equa
Explore this

GENDER EQUALITY

UNDERSTANDING THE INDICATORS

Marriage: assessing legal constraints related to marriage

Parental leave: the laws affecting women's work after having children

Assets: the gender differences in property and inheritance law

Entrepreneurship: the constraints women face when running a business

Wages: the constraints on women's freedom of movement

Workplace: the laws affecting women's decisions on pursuing a career

Pay: the laws and regulations concerning gender wage gap

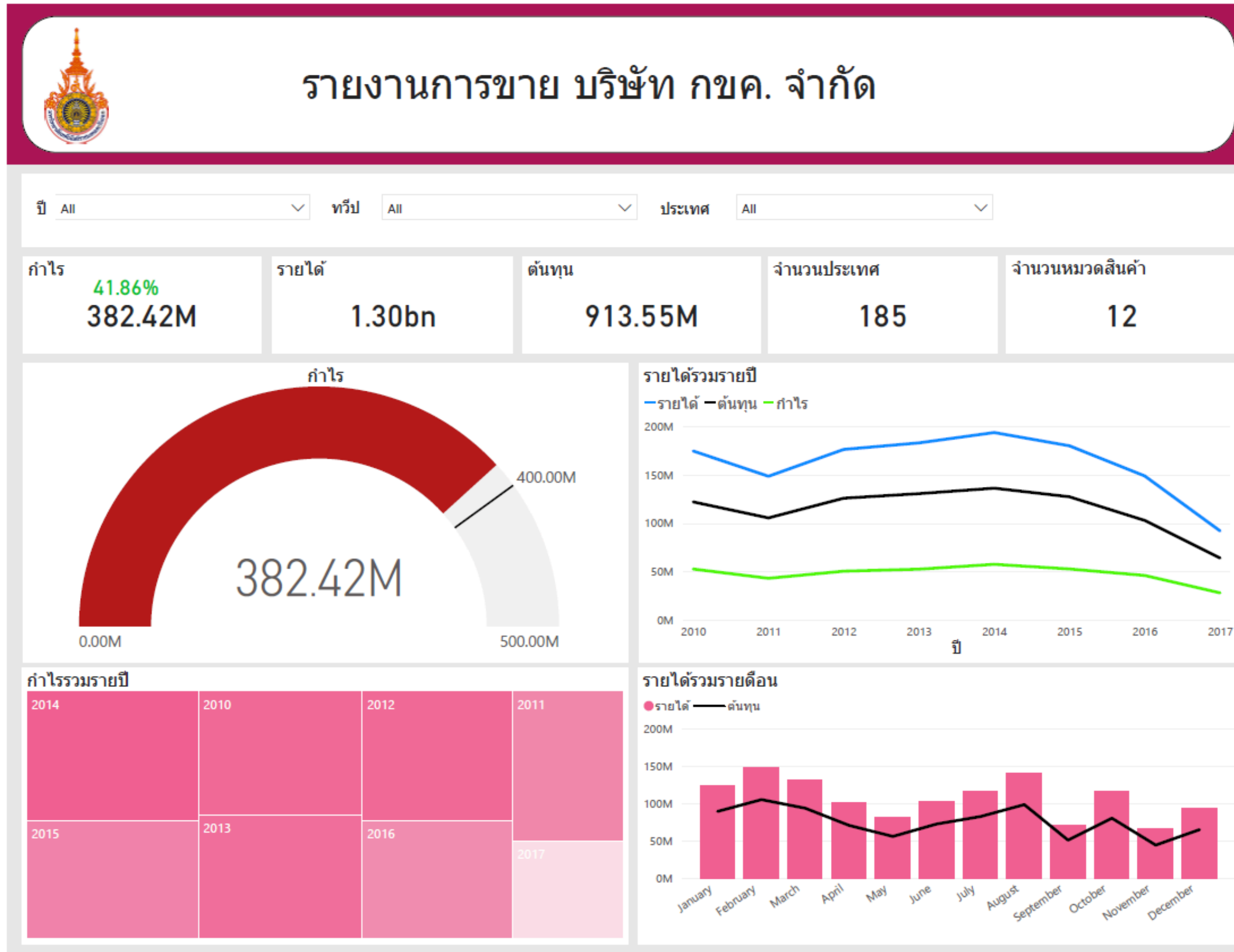
Pensions: the laws affecting the role of a woman's pension

Open

Recent Favorites My apps From external orgs

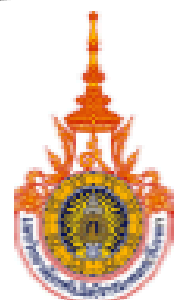
Filter by keyword Filter

- ตัวอย่างรายงานการขาย
product_sale.xlsx



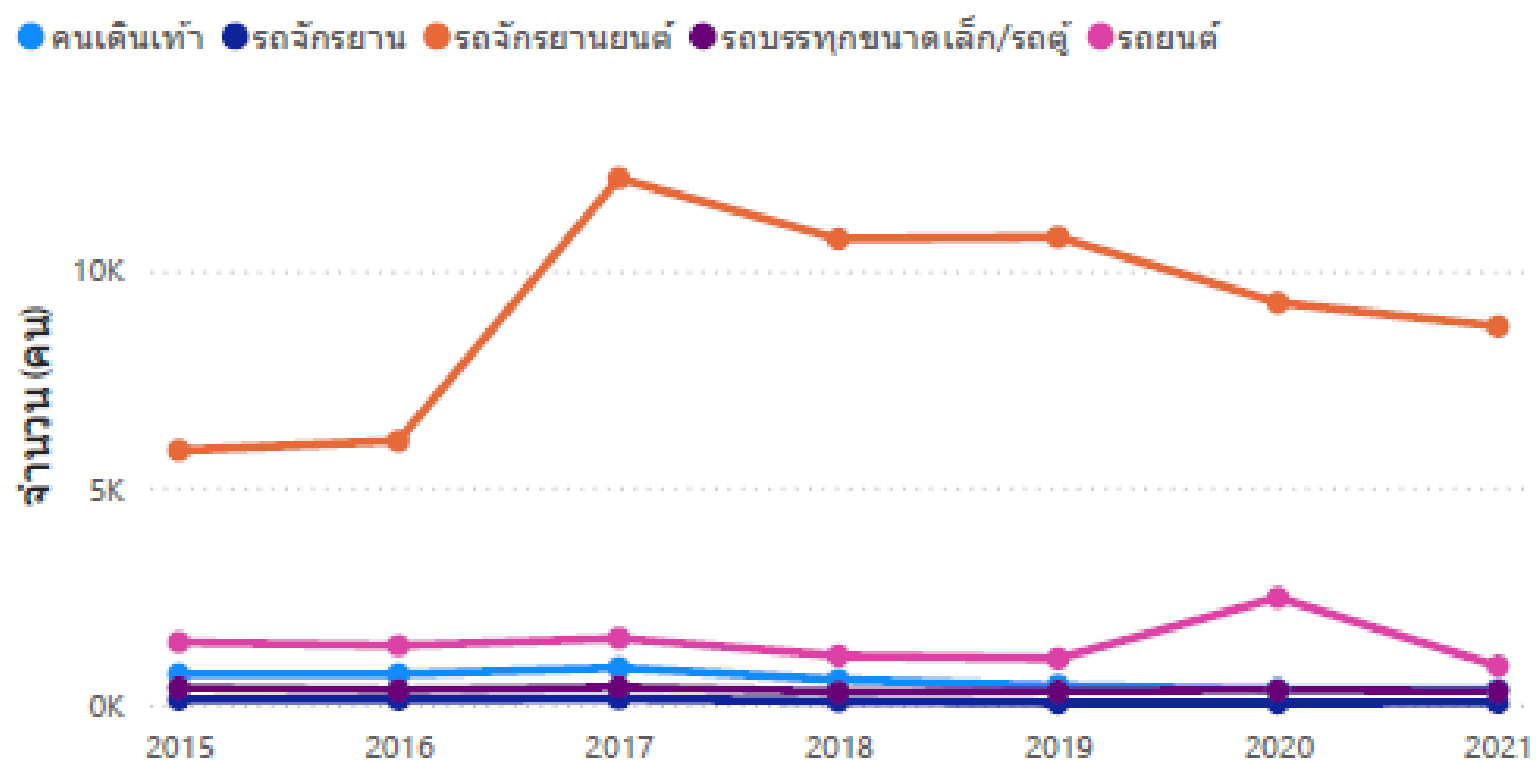
- ตัวอย่าง รายงานการเสียชีวิตจากอุบัติเหตุบนท้องถนน

accident.xlsx



รายงานการเสียชีวิตจากอุบัติเหตุบนท้องถนน

จำนวนผู้เสียชีวิตแยกตามยานพาหนะ



จังหวัดที่เกิดอุบัติเหตุมากที่สุด 10 อันดับแรก

All

จังหวัด	จำนวน (คน)
นครราชสีมา	6,292
กรุงเทพมหานคร	5,796
ชลบุรี	5,252
อุบลราชธานี	4,232
เชียงใหม่	4,146
ขอนแก่น	3,434
เชียงราย	3,201
ระยอง	3,150
นครศรีธรรมราช	2,941
บุรีรัมย์	2,914
Total	41,358

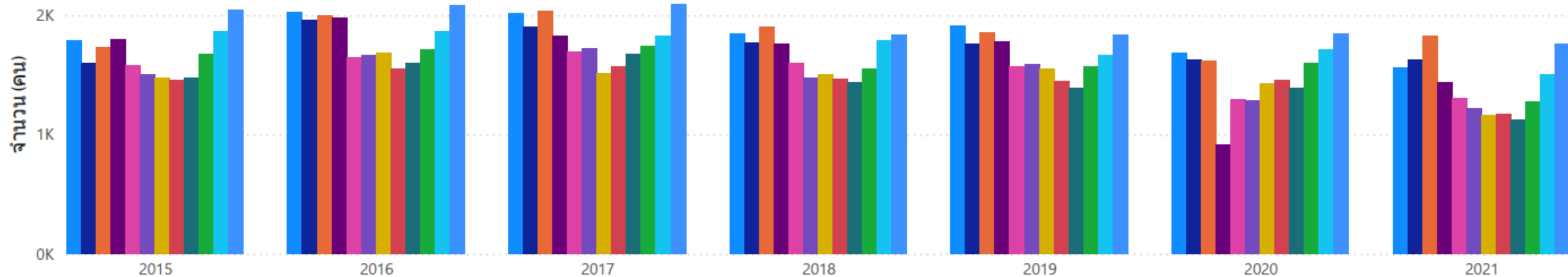
อายุ

ช่วงอายุ ▲	จำนวน (คน)
0-19	31,903
20-39	43,926
40-59	37,745
60 ปีขึ้นไป	24,361
Total	137,935

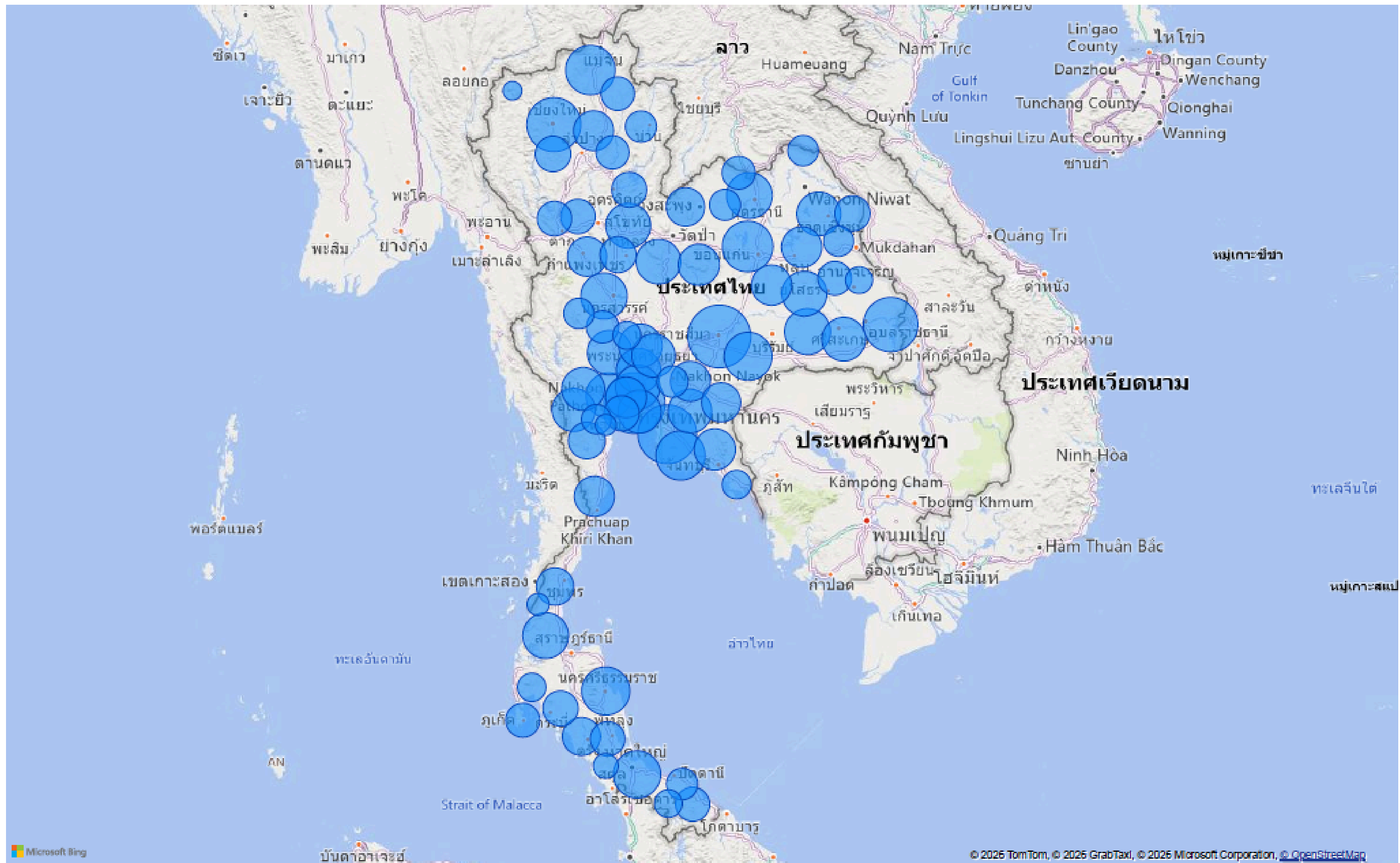
เพศ

เพศ ▼	จำนวน (คน)
ชาย	104,572
หญิง	28,001
อื่น ๆ	5,362
Total	137,935

January February March April May June July August September October November December



Month	2015	2016	2017	2018	2019	2020	2021	Total
January	1786	2023	2018	1841	1908	1679	1558	12813
February	1596	1956	1902	1772	1758	1628	1627	12239
March	1726	1999	2036	1904	1853	1614	1822	12954
April	1799	1973	1827	1763	1781	916	1433	11492
May	1574	1645	1688	1593	1570	1296	1307	10673
June	1499	1661	1723	1471	1584	1281	1220	10439
July	1477	1686	1509	1504	1548	1424	1162	10310
August	1456	1554	1568	1469	1443	1452	1173	10115
September	1473	1598	1671	1437	1393	1388	1118	10078
October	1672	1708	1742	1550	1570	1602	1277	11121
November	1860	1860	1827	1788	1662	1707	1504	12208
December	2042	2082	2096	1839	1834	1844	1756	13493
Total	19960	21745	21607	19931	19904	17831	16957	137935



5.6 แบบฝึกหัด

1. ให้วิเคราะห์ข้อมูล Diamonds (diamonds.xlsx) หรือ Titanic (titanic.xlsx) อย่างใดอย่างหนึ่ง แล้วตอบคำถามต่อไปนี้

1.1 หาค่า Statistics เช่น

- แนวโน้มเข้าสู่ส่วนกลาง
- การกระจายตัว
- การแจกแจงข้อมูล
- ดู correlation (เช่น Pearson, Chi-square, T-test)

1.2 ทำ Data Visualization เช่น

- Scatter Plot
- Bar Chart

1.3 สรุปผล และ บอกว่าเหตุผลว่า ตัวแปรไหนสำคัญ ต่อการนำไปสร้างโมเดล

Note : ข้อมูล Titanic พิจารณาเฉพาะตัวแปร survived, class, sex, who, alone, และ age

2. ให้วิเคราะห์ข้อมูลราคาที่ดิน แล้วตอบคำถามต่อไปนี้

2.1 หาค่า Statistics เช่น

- แนวโน้มเข้าสู่ส่วนกลาง
- ดูการกระจายตัว
- ดูการแจกแจงข้อมูล
- ดู correlation (เช่น Pearson, Chi-square, T-test)

2.2 ทำ Data Visualization เช่น

- Scatter Plot
- Bar Chart

2.3 สรุปผล และ บอกว่าเหตุผลว่า ตัวแปรไหนสำคัญต่อการนำไปสร้างโมเดล

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	50	5	250
2	100	4	240
3	80	5	260
4	200	3	200
5	500	1	150
6	300	2	180
7	700	0	130
8	600	1	140
9	250	3	190
10	120	4	230

3. ให้วิเคราะห์ข้อมูลโรคเบาหวาน แล้วตอบคำถามต่อไปนี้

3.1 หาค่า Statistics เช่น

- แนวโน้มเข้าสู่ส่วนกลาง
- การกระจายตัว
- การแจกแจงข้อมูล
- ดู correlation (เช่น Pearson, Chi-square, T-test)

3.2 ทำ Data Visualization เช่น

- Scatter Plot
- Bar Chart

3.3 สรุปผล และ บอกว่าเหตุผลว่า ตัวแปรไหนสำคัญต่อการนำไปสร้างโมเดล

รายการ	อายุ (ปี)	ปริมาณน้ำตาล ในเลือด (mg/dL)	การเป็นโรค
1	25	85	ไม่เป็นโรค
2	30	90	ไม่เป็นโรค
3	45	130	เป็นโรค
4	35	120	เป็นโรค
5	50	150	เป็นโรค
6	23	88	ไม่เป็นโรค
7	60	160	เป็นโรค
8	55	155	เป็นโรค
9	40	135	ไม่เป็นโรค
10	28	95	ไม่เป็นโรค

4. ให้วิเคราะห์ข้อมูลสภาพอากาศ แล้วตอบคำถามต่อไปนี้

4.1 หาค่า Statistics เช่น

- แนวโน้มเข้าสู่ส่วนกลาง
- ดูการกระจายตัว
- ดูการแจกแจงข้อมูล
- ดู correlation (เช่น Pearson, Chi-square, T-test)

4.2 ทำ Data Visualization เช่น

- Scatter Plot
- Bar Chart

4.3 สรุปผล และ บอกว่าเหตุผลว่า ตัวแปรไหนสำคัญ ต่อการนำไปสร้างโมเดล

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	ความเร็วลม (km/h)	ความชื้นสัมพัทธ์ (%)	PM 2.5 (µg/m³)
1/2/2569	35	18	16.4	69.5	22
2/2/2569	34	15.2	14.9	65.3	25
3/2/2569	33	8.5	14.1	57	28
4/2/2569	34	2	15.4	47.7	26
5/2/2569	35	0	15.3	46.1	24
6/2/2569	32	12.3	12.9	60.2	30
7/2/2569	31	10.4	12.6	56.6	33
8/2/2569	29	0.5	9.8	44.2	39
9/2/2569	30	0	9.6	42.7	38
10/2/2569	28	10.7	11.7	56.2	34



Thank you
For Your Attention



สาขาวิชาวิทยาการคอมพิวเตอร์
คณะบริหารธุรกิจและเทคโนโลยีสารสนเทศ 